

POLITECNICO DI TORINO

Laurea Magistrale in Ingegneria Matematica

Numerical optimization for large scale problems

Unconstrained Optimization – Derivative–based Optimization



Bagnato Francesca	346710
Foni Gianmarco	347952
Maccarone Giovanna	349134

Contents

1	Introduction	2
2	Modified Newton Method	2
2.1	Hessian regularization and Armijo backtracking	2
3	Truncated Newton Method	3
3.1	Newton–CG with Armijo backtracking	3
4	Test problems	4
4.1	Problem 16: Banded Trigonometric	4
4.2	Problem 31: Broyden tridiagonal least-squares	4
5	Parameter tuning and implementation choices	5
6	Finite Differences Implementation	5
6.0.1	Case 1: Exact gradient and finite-difference Hessian	6
6.0.2	Case 2: Fully finite-difference derivatives	6
7	Numerical Results	6
7.1	Problem 16: Banded Trigonometric (Trig16)	6
7.2	Problem 31: Broyden Tridiagonal (Broyden31)	7
A	Code	13
B	Tables	26

1 Introduction

This report presents a numerical study on two derivative-based methods for unconstrained optimization: the Modified Newton method and the Truncated Newton method. Both algorithms are implemented in MATLAB and integrated with a back-tracking line search strategy to ensure global convergence. The methods are tested on two benchmark problems from [1], namely Problem 16 and Problem 31. The goal is to assess robustness and scalability when the dimension grows ($n = 2, 10^3, 10^4, 10^5$), considering both exact derivatives and finite-difference (FD) approximations. For each test problem we follow the assignment protocol: we use the suggested starting point $\bar{x}$, and five additional starting points sampled uniformly in the hypercube $[\bar{x}_1 - 1, \bar{x}_1 + 1] \times \cdots \times [\bar{x}_n - 1, \bar{x}_n + 1]$ with a fixed random seed equal to the minimum student ID in the team. Performance is compared in terms of iterations, computational time, final gradient norm, success/failure with respect to a tolerance on $\|\nabla f(x_k)\|$, and (when required) experimental convergence rates, using problem-dependent values of the maximum number of iterations.

Finally, we extend the study to the finite-difference setting using an efficient implementation that exploits the sparsity/banded structure of the derivatives, aiming to reduce the number of function/gradient evaluations with respect to general-purpose FD schemes.

2 Modified Newton Method

2.1 Hessian regularization and Armijo backtracking

To solve unconstrained nonlinear optimization problems in which the Hessian matrix may be indefinite or poorly conditioned, we implement a Modified Newton method. The aim is to retain the fast local convergence properties of Newton-type methods while ensuring robustness and global convergence.

At iteration k , given the current iterate x_k , we compute

$$f_k = f(x_k), \quad g_k = \nabla f(x_k), \quad H_k = \nabla^2 f(x_k).$$

Since H_k is not guaranteed to be symmetric positive definite, its properties are verified at each iteration. If H_k is found to be non-positive definite, a regularized Hessian matrix is constructed as

$$B_k = H_k + E_k,$$

where $E_k = \tau_k I$ and $\tau_k \geq 0$ is a correction parameter chosen to enforce positive definiteness.

The initial value of τ_k is selected based on the diagonal entries of H_k . If all diagonal elements are strictly positive, τ_k is initially set to zero. Otherwise, it is chosen as

$$\tau_k = \beta - \min(\text{diag}(H_k)),$$

where $\beta > 0$ is a safeguard parameter. In general, we want B_k to stay as close as possible to our initial Hessian matrix. Therefore, we aim to choose the smallest possible correction parameter τ_k to achieve this, but as big as needed to make B_k positive definite.

Positive definiteness of B_k is then verified by attempting a Cholesky factorization. If the factorization fails, τ_k is iteratively increased according to

$$\tau_k = \max(\beta, 2\tau_k)$$

until a successful factorization is obtained or a maximum number of attempts is reached.

Once a symmetric positive definite matrix B_k is available, the Modified Newton direction p_k is computed by solving

$$B_k p_k = -g_k.$$

Using the Cholesky factorization $B_k = R^\top R$, the direction is obtained via two triangular solves. Since B_k is symmetric positive definite and $g_k \neq 0$, the resulting direction satisfies

$$g_k^\top p_k = -g_k^\top B_k^{-1} g_k < 0,$$

and is therefore a descent direction.

To ensure global convergence, the step length α_k is selected by means of an inexact line search based on Armijo backtracking. Starting from $\alpha = 1$, the step length is iteratively reduced according to $\alpha \leftarrow \rho \alpha$, with $0 < \rho < 1$, until the sufficient decrease condition

$$f(x_k + \alpha p_k) \leq f(x_k) + c_1 \alpha g_k^\top p_k, \quad 0 < c_1 < 1,$$

is satisfied or a maximum number of backtracking steps is reached.

The algorithm is stopped when the gradient norm satisfies

$$\|g_k\| \leq \text{tolgrad},$$

or when the maximum number of iterations `kmax` is attained.

Implementation notes. To keep the algorithm efficient for large-scale problems, we handle the Hessian matrix in a sparse format and only inspect its diagonal when initializing the regularization. For structures like the ones which characterize the particular problems faced in this project, this approach allows sparse Cholesky factorizations to remain fast and feasible even as dimensions grow.

3 Truncated Newton Method

3.1 Newton–CG with Armijo backtracking

The Truncated Newton method is designed for large-scale optimization problems where forming or factorizing the Hessian matrix is computationally expensive. The method avoids the direct solution of the Newton system by computing an approximate search direction through a Conjugate Gradient (CG) iteration.

At the current iterate x_k , we compute

$$f_k = f(x_k), \quad g_k = \nabla f(x_k),$$

and consider the Newton system

$$\nabla^2 f(x_k) p_k = -g_k.$$

The algorithm operates through a two-level nested structure: the outer loop updates the current point x_k by managing convergence and backtracking strategies, while the inner loop computes an approximate descent direction p_k by solving the system above using the Conjugate Gradient method. The CG process starts from the zero vector, producing a sequence of increasingly accurate approximate solutions.

The accuracy of the inner CG solve is controlled by a forcing term

$$\eta_k = \min\{0.5, \sqrt{\|\nabla f(x_k)\|}\},$$

which allows for inexact solves far from the solution and increasingly accurate solves as convergence is approached.

During the inner CG loop, we monitor the curvature of the objective function, defined by the term $d_j^\top H_k d_j$. This value describes how the function behaves along the current search direction. If $d_j^\top H_k d_j > 0$, the curvature along that direction is positive. The CG method can safely proceed to refine the direction p_k . In the other hand, if the algorithm encounters $d_j^\top H_k d_j \leq 10^{-12}$, it detects that the function is flat or non-convex. In this case, the inner loop stops immediately. As implemented in the code, if this happens at the very first iteration, we fall back to the steepest descent direction $p_k = -g_k$ to guarantee that we always move in a direction that decreases the function.

Once a direction p_k is obtained, the step length α_k is chosen using an Armijo backtracking line search, starting from $\alpha = 1$ and enforcing sufficient decrease of the objective function.

The algorithm terminates when

$$\|g_k\| \leq \text{tolgrad},$$

or when the maximum number of outer iterations `kmax` is reached.

Implementation notes. One of the main advantages of this method is that we never actually need to build or store the full Hessian matrix. Instead, the algorithm only interacts with it through matrix-vector products during the CG steps. This makes the approach much lighter and perfect for handling very large problems where memory is limited. To keep track of how hard the algorithm is working, we record the number of inner CG iterations at every step; this gives us a clear picture of the total computational effort required to find the solution.

4 Test problems

4.1 Problem 16: Banded Trigonometric

The Banded Trigonometric problem is defined by the objective function

$$F(x) = \sum_{i=1}^n i(1 - \cos x_i) + 2 \sum_{i=1}^{n-1} \sin x_i - (n-1) \sin x_n,$$

with standard starting point $\bar{x} = \mathbf{1}$. The gradient of the objective function is available in closed form. Its components are given by

$$(\nabla f(x))_i = i \sin(x_i) + 2 \cos(x_i), \quad i = 1, \dots, n-1,$$

while the last component accounts for the boundary term and reads

$$(\nabla f(x))_n = n \sin(x_n) - (n-1) \cos(x_n).$$

The Hessian matrix is diagonal. Its diagonal entries are given by

$$(\nabla^2 f(x))_{ii} = i \cos(x_i) - 2 \sin(x_i), \quad i = 1, \dots, n-1,$$

and

$$(\nabla^2 f(x))_{nn} = n \cos(x_n) + (n-1) \sin(x_n).$$

The function is smooth and nonconvex, with trigonometric nonlinearities that introduce multiple local minima and saddle points.

A key feature of this problem is that its Hessian matrix, as stated before, is diagonal, with entries depending on trigonometric functions of the components of x . As a consequence, the Hessian may change sign depending on the current iterate, and is not guaranteed to be positive definite along the optimization path. This makes the problem particularly suitable for testing Hessian modification strategies in the Modified Newton method and curvature detection mechanisms in the Truncated Newton method.

The problem is built to be efficient: both the gradient and the Hessian only require $\mathcal{O}(n)$ operations and can be stored using sparse formats. This efficiency allows us to test our methods on very large scales, reaching up to $n = 10^5$, where how an algorithm scales and handles numerical stress really matters. Since the Hessian is mostly diagonal, we expect the Modified Newton method to converge very quickly once it ensures the matrix is positive definite. On the other hand, the Truncated Newton method is more flexible: it provides fast convergence when possible, but naturally slows down to behave like a simpler first-order method if the curvature information becomes unreliable or messy.

4.2 Problem 31: Broyden tridiagonal least-squares

The Broyden tridiagonal problem is a classical example of a nonlinear least-squares problem. The objective function has the form

$$f(x) = \frac{1}{2} \sum_{k=1}^n r_k(x)^2,$$

where the residuals are defined as

$$r_k(x) = (3 - 2x_k)x_k - x_{k-1} - 2x_{k+1} + 1,$$

with boundary conditions $x_0 = x_{n+1} = 0$. The standard starting point provided in the assignment is adopted, namely $x^{(0)} = (-1, \dots, -1)$.

Each residual depends nonlinearly on the variable x_k , which introduces genuine nonlinearity into the problem, while the coupling with only neighboring components results in a sparse tridiagonal structure.

Thanks to its least-squares formulation, the gradient of the objective function can be written as

$$\nabla f(x) = J(x)^\top r(x),$$

where $J(x)$ denotes the Jacobian matrix of the residual vector. The Hessian matrix has the form

$$\nabla^2 f(x) = J(x)^\top J(x) - 4 \operatorname{diag}(r(x)),$$

and is therefore sparse but generally indefinite, due to the contribution of the residual-dependent term.

These properties make the Broyden problem particularly suitable for testing second-order methods on large-scale instances. The computational cost per iteration grows linearly with the dimension, allowing experiments to be performed efficiently up to $n = 10^5$.

5 Parameter tuning and implementation choices

The algorithmic parameters were selected through preliminary numerical experiments aimed at balancing robustness and computational efficiency across different dimensions. The stopping tolerance on the gradient norm was set to $\text{tolgrad} = 10^{-6}$, here is not considerate as a free parameter but instead as an hard constraint. The remaining parameters, some shared between the Modified and Truncated algorithms, such as **btmax** (maximum number of backtracking iterations), **c1** (Armijo condition), **kmax** (maximum number of outer iterations), **rho** (rescaling factor in backtracking), and problem-specific ones, namely **beta** (Hessian correction term) and **max_cg** (maximum number of CG iterations for Truncated), were tuned using a combination of successive refinements and a grid search.

Initially we generated some baseline configuration combining discrete levels of each parameter, and tested it with multiple runs on random initial points. If a run failed to converge, parameter levels were adaptively adjusted according to a predefined escalation logic (prioritizing **btmax**, then **max_cg**, then **kmax**), and the updated configuration was reinserted into the testing queue. This process continued until all configurations had been evaluated, resulting in a set of admissible and robust configurations. During the second refinement stage, these admissible configurations were evaluated more rigorously with repeated runs for problem sizes 10^3 and 10^4 . Configurations that failed were discarded. For each surviving configuration, a loss function was computed, defined as

$$\mathcal{L} = w_t \cdot t_{\text{run}} + w_g \cdot \max(0, \log_{10}(\|\nabla f\|/\text{tolgrad}))$$

where t_{run} is the median run time over repeated starts, $\|\nabla f\|$ is the final gradient norm, and k is the number of outer iterations. Weights w_t and w_g , balance the relative importance of runtime and gradient accuracy. The top 10% of configurations with the lowest loss were selected, and the best among them was chosen as the final configuration. This approach ensures robust configurations that scale reasonably with problem size. Moreover, it is constructive: parameters such as **btmax**, **max_cg**, and **kmax** act as safeguards, assuming values subordinate to the problem's specific requirements.

Table 1: Best parameter configurations for Modified and Truncated Newton methods on Trig16 and Broyden31 problems

Method	Trig16	Broyden31
Modified Newton	btmax = 10	btmax = 5
	c1 = $1e-4$	c1 = $1e-4$
	kmax = 50	kmax = 20
	rho = 0.5	rho = 0.5
	beta = $1e-2$	beta = $1e-3$
Truncated Newton	btmax = 40	btmax = 5
	c1 = $1e-3$	c1 = $1e-4$
	kmax = 50	kmax = 20
	rho = 0.8	rho = 0.3
	max_cg = 1000	max_cg = 5

6 Finite Differences Implementation

In this following section, we investigate how the performance of second-order optimization methods is affected when derivatives are approximated by finite differences (FD). The main goal is to assess robustness with respect to the choice of the FD step size and to compare a fixed increment with a componentwise scaled increment.

For each differentiation we consider a standard fixed step

$$h = 10^{-k}, \quad k \in \{4, 8, 12\},$$

as well as a scaled step defined componentwise as

$$h_i = 10^{-k} \max(|x_i|, 1), \quad i = 1, \dots, n.$$

For both test problems (Trig16 and Broyden31) and for each dimension $n \in \{2, 10^3, 10^4, 10^5\}$, the methods are tested from six different starting points: the standard reference point $\bar{x}$ and five random points sampled near it. In our implementation, we use finite differences to approximate the derivatives, using forward or central schemes

depending on the quantity being approximated.

We distinguish between two main testing scenarios: the one in which we use the analytical formula for the gradient while approximating only the Hessian matrix and the other one in which both the gradient and the Hessian are computed using finite differences.

6.0.1 Case 1: Exact gradient and finite-difference Hessian

In this first scenario, we use the exact formula for the gradient and approximate only the Hessian matrix. This allows us to see how small errors in the second derivatives affect the performance of our methods. To approximate the Hessian, we use the forward-difference formula based on the gradient:

$$H(:, j) \approx \frac{\nabla f(x + h_j e_j) - \nabla f(x)}{h_j},$$

where we perturb the j -th component of x by a small step h_j . Since we are dealing with very large problems, calculating every single entry of the Hessian would make the algorithm too slow. Instead, we take advantage of the fact that our matrices are sparse: for the Trig16 problem, the Hessian is diagonal while for the Broyden31 problem, it is pentadiagonal.

To speed things up, a simple one dimensional coloring strategy is adopted, grouping columns by stride $m = 2 \text{bw} + 1$ and perturbing multiple coordinates simultaneously. This reduces the number of gradient evaluations per Hessian, which is crucial for largescale problems.

The numerical results reported for this case compare convergence behavior, iteration counts, and computational times with respect to the baseline implementation using exact Hessians.

6.0.2 Case 2: Fully finite-difference derivatives

In the second scenario, both gradient and Hessian are approximated numerically. This is the most realistic scenario when analytic derivatives are unavailable, but also the most sensitive to FD noise.

To keep things stable, we use different strategies depending on the problem:

For Trig16, we exploit the separable structure of the objective function. Due to the diagonal structure of the Hessian, we calculate the derivatives one component at a time. We use central differences applied componentwise, which take a bit more effort than forward differences but are much more accurate. This results in a clean, diagonal Hessian.

For Broyden31, which is formulated as a nonlinear least squares problem, we do not approximate the Hessian directly. Instead, we approximate the Jacobian of the residuals using central differences. Then we build the gradient and the Hessian as follows:

$$\nabla f(x) \approx J(x)^\top r(x), \quad \nabla^2 f(x) \approx J(x)^\top J(x).$$

This corresponds to a Gauss-Newton approximation of the Hessian. It is much more stable than trying to differentiate a noisy gradient, and it has a huge advantage: the resulting Hessian is always symmetric and positive semidefinite by construction.

To improve efficiency, we avoid redundant calculations by reusing the Jacobian estimate for both the gradient and the Hessian within the same iteration: the gradient and Hessian evaluations at the same iterate are cached, so that the Jacobian estimate is reused within each iteration.

7 Numerical Results

The performance of the Modified Newton (MN) and Truncated Newton (TN) methods was evaluated on two benchmark problems, namely Trig16 and Broyden31. The experiments were carried out for dimensions $n \in \{2, 10^3, 10^4, 10^5\}$, considering both exact derivatives and finite-difference (FD) approximations. Two FD settings were analyzed: Case 1, where the gradient is exact and the Hessian is approximated, and Case 2, where all derivatives are computed numerically. The stopping criterion was $\|\nabla f(x_k)\| < 10^{-6}$, while the maximum number of outer iterations was chosen according to the tuning strategy described in Section 5.

7.1 Problem 16: Banded Trigonometric (Trig16)

Exact derivatives. Table 2 summarizes the numerical results obtained for Trig16 using exact derivatives. For small and medium dimensions ($n \leq 10^4$), the Modified Newton method consistently converges in a small number

of iterations from all starting points, achieving very small gradient norms and experimental convergence rates close to the theoretical quadratic behavior. The Truncated Newton method requires more iterations, but remains robust and exhibits a nearly linear convergence rate.

For $n = 10^5$, a clear difference emerges between the two methods. Modified Newton often reaches the maximum number of iterations when started from random initial points, indicating a loss of robustness in very large-scale settings. In contrast, the Truncated Newton method successfully converges from all initializations, confirming its suitability for large-scale problems where the explicit factorization of the Hessian becomes computationally demanding.

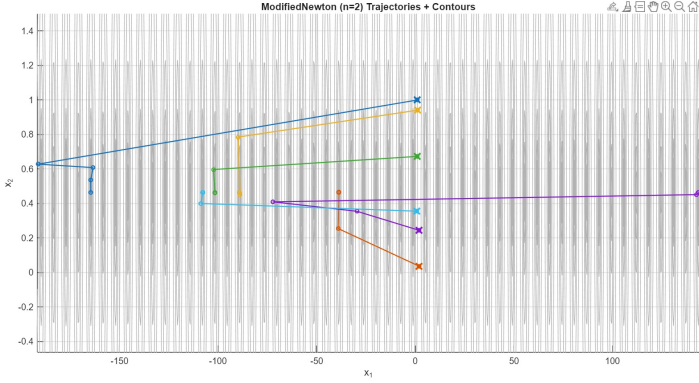


Figure 1: Modified Newton trajectories for the Trig16 problem with $n = 2$ using exact derivatives.

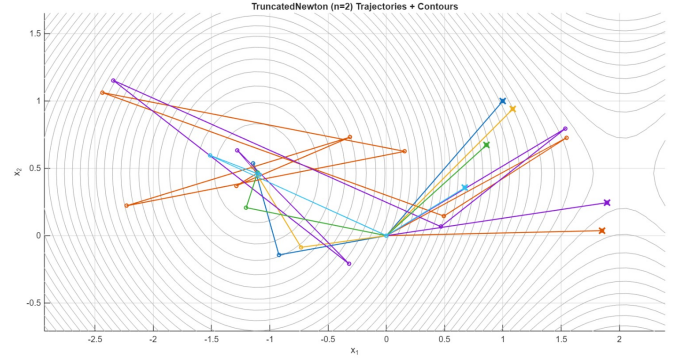


Figure 2: Truncated Newton trajectories for the Trig16 problem with $n = 2$ using exact derivatives.

As shown in Figure 1, Modified Newton trajectories rapidly align with the curvature of the objective function and become almost straight close to the solution, which is consistent with second-order local convergence. Figure 2 shows that Truncated Newton follows more conservative paths, especially far from the minimizer, due to the inexact solution of the Newton system.

Finite differences. The introduction of finite-difference approximations significantly affects the numerical behavior of Trig16. In Case 1, with step size $h = 10^{-8}$, Modified Newton preserves the qualitative convergence behavior for $n = 2$, although small oscillations appear near the solution as a consequence of inaccurate curvature information. Truncated Newton is less sensitive to Hessian approximation errors and remains stable.

In Case 2, where both gradient and Hessian are approximated, Modified Newton frequently fails to converge even for moderate dimensions, typically reaching backtracking or iteration limits. Truncated Newton exhibits slightly better robustness, but convergence is still rarely achieved as the dimension increases.

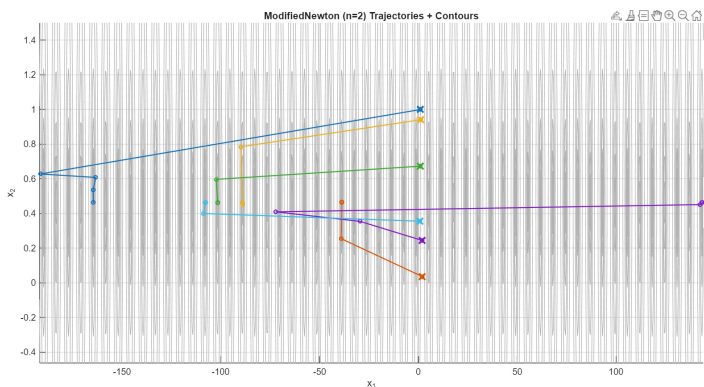


Figure 3: Modified Newton trajectories for Trig16 with $n = 2$ using finite differences ($h = 10^{-8}$).

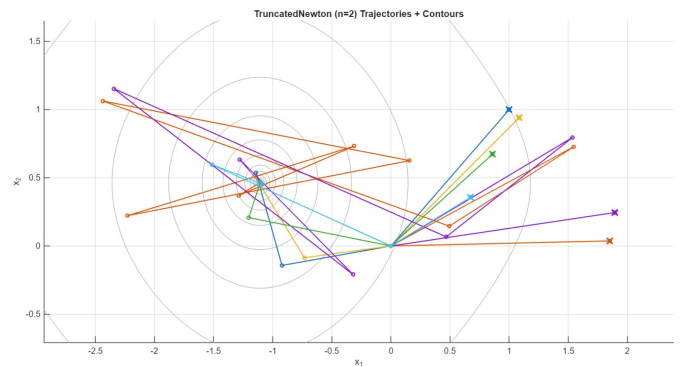


Figure 4: Truncated Newton trajectories for Trig16 with $n = 2$ using finite differences ($h = 10^{-8}$).

7.2 Problem 31: Broyden Tridiagonal (Broyden31)

Exact derivatives. The numerical results for Broyden31 with exact derivatives are reported in Table 3. Both methods show robust convergence for all tested dimensions up to $n = 10^5$. Modified Newton typically converges in fewer iterations and achieves experimental convergence rates close to quadratic, benefiting from efficient sparse Cholesky factorizations. Truncated Newton requires more iterations, but its computational cost grows mildly with the dimension due to the use of matrix-vector products within the inner CG iterations.

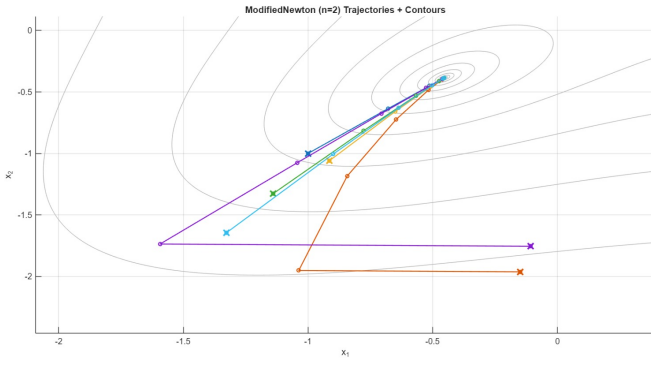


Figure 5: Modified Newton trajectories for the Broyden31 problem with $n = 2$ using exact derivatives.

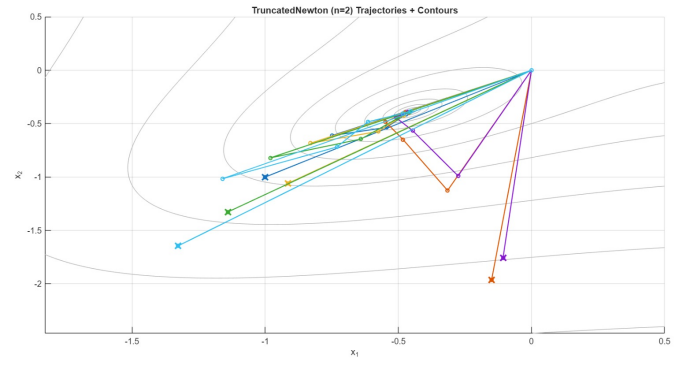


Figure 6: Truncated Newton trajectories for the Broyden31 problem with $n = 2$ using exact derivatives.

In Figure 5, Modified Newton rapidly converges once the iterates enter the narrow curved valley of the objective function. The Truncated Newton trajectories shown in Figure 6 are smoother and more conservative, reflecting the effect of curvature detection and truncated inner solves.

Finite differences. When finite differences are employed, Broyden31 proves significantly more robust than Trig16. In Case 1, both methods preserve convergence for moderate and large dimensions, although the number of iterations increases. In Case 2, convergence is still achieved in several configurations, especially for Truncated Newton, whose inexact nature makes it more tolerant to derivative noise.

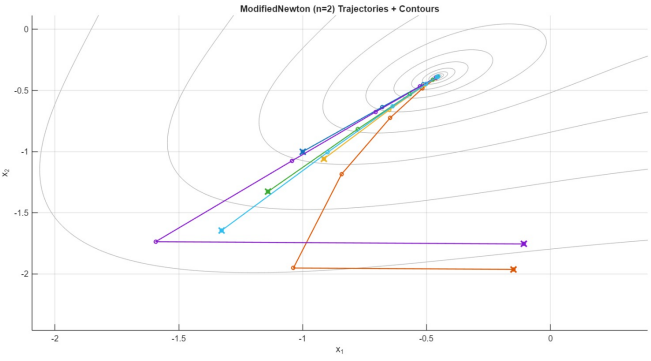


Figure 7: Modified Newton trajectories for Broyden31 with $n = 2$ using finite differences ($h = 10^{-8}$).

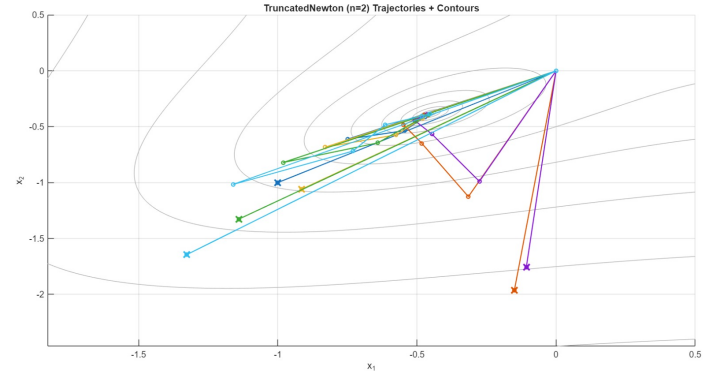
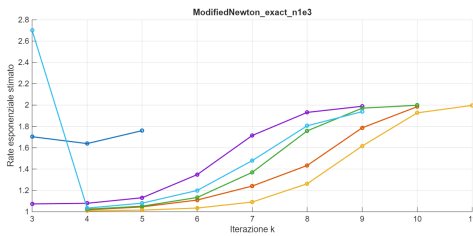
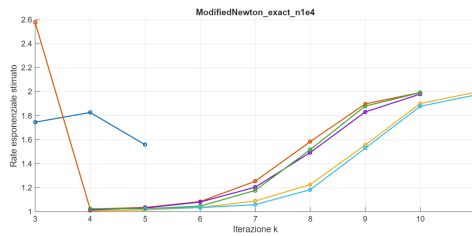


Figure 8: Truncated Newton trajectories for Broyden31 with $n = 2$ using finite differences ($h = 10^{-8}$).

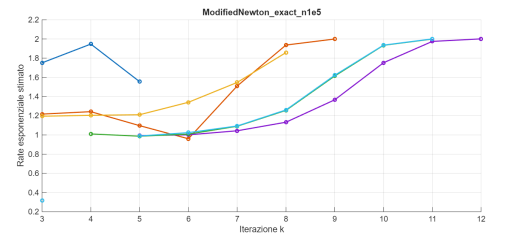
Overall, the results confirm that Modified Newton achieves fast convergence when accurate derivatives are available, while Truncated Newton offers superior robustness and scalability in large-scale and finite-difference settings, in full agreement with the expected theoretical behavior.



(a) Est. Rate of Convergence, Modified on Broyden31 $N = 1e3$

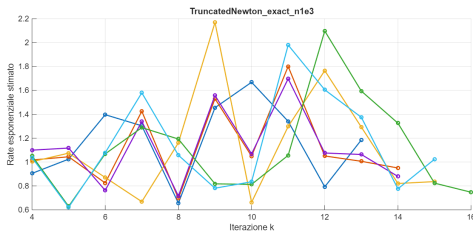


(b) Est. Rate of Convergence, Modified on Broyden31 $N = 1e4$

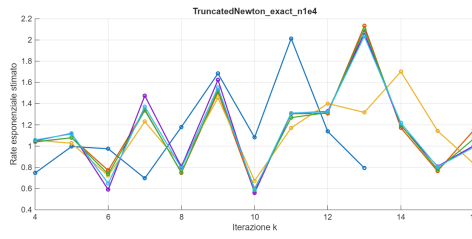


(c) Est. Rate of Convergence, Modified on Broyden31 $N = 1e5$

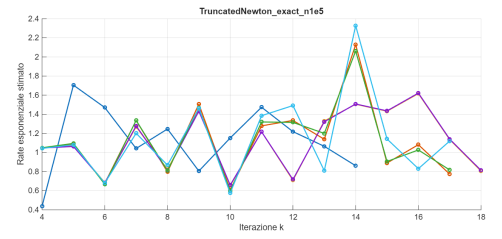
Figure 9: Experimental convergence rates for the Modified Newton method on Broyden31, we can see it seems to reach an experimental rate = 2 which is expected for our implementation of the Modified method. *é il nostro risultato migliore, parte con rate lineare e poi quando è vicino al minimo converge con rate quadratico come ci aspettiamo*



(a) Est. Rate of Convergence, Truncated on Broyden31 $N = 1e3$

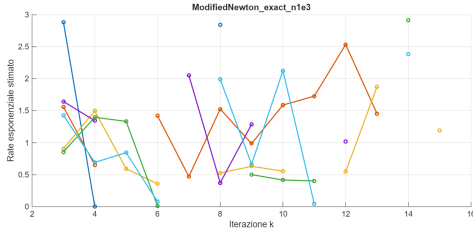


(b) Est. Rate of Convergence, Truncated on Broyden31 $N = 1e4$

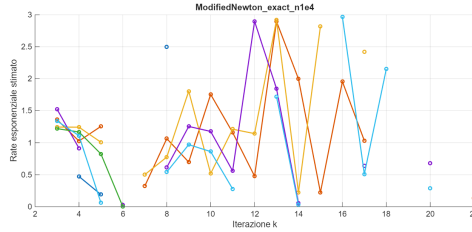


(c) Est. Rate of Convergence, Truncated on Broyden31 $N = 1e5$

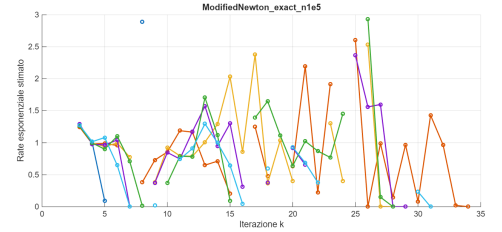
Figure 10: Experimental convergence rates for the Truncated Newton method on Broyden31, we can see it seems to oscillate around an experimental rate of 1.2



(a) Est. Rate of Convergence, Modified on Trig16 $N = 1e3$

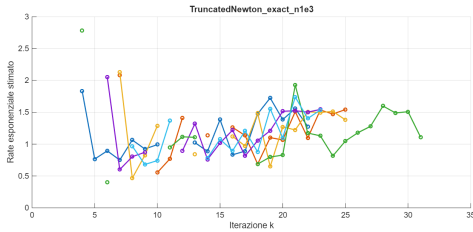


(b) Est. Rate of Convergence, Modified on Trig16 $N = 1e4$

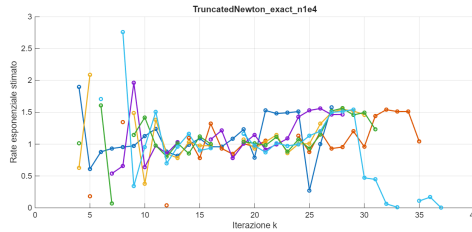


(c) Est. Rate of Convergence, Modified on Trig16 $N = 1e5$

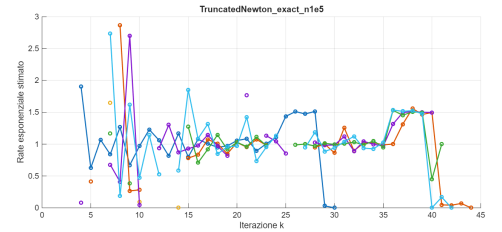
Figure 11: Experimental convergence rates for the Modified Newton method on Trig16. We are not always able to compute the rate due to **computational issues in the estimate** Is worth to notice this pattern: NON QUI il modified nel problema trig16 con le prime iterazioni si sposta in un minimo locale molto distante e poi usa le successive iterazioni per spostarsi nella zona del minimo in punti ugualmente distanti dal minimo teorico, il truncated invece esplora subito spazio più vicino e riesce a convergere con un rate migliore, parte con rate lineare e dopo sembra convergere ad un rate 1.2 1.5



(a) Est. Rate of Convergence, Truncated on Trig16 $N = 1e3$



(b) Est. Rate of Convergence, Truncated on Trig16 $N = 1e4$



(c) Est. Rate of Convergence, Truncated on Trig16 $N = 1e5$

Figure 12: Experimental convergence rates for the Truncated Newton method on Trig16 we can see it seems to oscillate around an experimental rate of 1.2

Table 2: Modified Newton vs Truncated Newton with exact derivatives on Problem 16 (Trig16).

Dim	x_0	Modified Newton						Truncated Newton					
		iter	$\ \nabla f(x_k)\ $	success	flag	rate conv.	time (s)	iter	$\ \nabla f(x_k)\ $	success	flag	rate (exp.)	time (s)
2	1	5	$3.23 \cdot 10^{-12}$	yes	—	2.99	0.00	7	$6.17 \cdot 10^{-11}$	yes	—	1.99	0.00
2	2	3	$5.63 \cdot 10^{-7}$	yes	—	2.91	0.00	8	$9.93 \cdot 10^{-7}$	yes	—	1.97	0.00
2	3	4	$1.02 \cdot 10^{-9}$	yes	—	—	0.00	7	$1.30 \cdot 10^{-9}$	yes	—	1.99	0.00
2	4	6	$1.88 \cdot 10^{-7}$	yes	—	—	0.00	8	$3.99 \cdot 10^{-8}$	yes	—	1.99	0.00
2	5	4	$4.20 \cdot 10^{-11}$	yes	—	—	0.00	7	$9.30 \cdot 10^{-8}$	yes	—	1.99	0.00
2	6	4	$1.21 \cdot 10^{-8}$	yes	—	—	0.00	8	$1.69 \cdot 10^{-10}$	yes	—	1.99	0.00
Avg (successes)		4	$1.27 \cdot 10^{-7}$	—	—	0.98	0.00	8	$1.88 \cdot 10^{-7}$	—	—	1.99	0.00
10^3	1	8	$1.53 \cdot 10^{-13}$	yes	—	2.84	0.00	14	$7.58 \cdot 10^{-7}$	yes	—	0.23	0.00
10^3	2	15	$4.79 \cdot 10^{-9}$	yes	—	2.03	0.00	15	$8.55 \cdot 10^{-7}$	yes	—	0.35	0.00
10^3	3	13	$6.92 \cdot 10^{-9}$	yes	—	2.33	0.00	15	$9.17 \cdot 10^{-7}$	yes	—	0.24	0.00
10^3	4	12	$7.95 \cdot 10^{-9}$	yes	—	2.11	0.00	15	$9.61 \cdot 10^{-7}$	yes	—	0.16	0.00
10^3	5	19	$2.85 \cdot 10^{-9}$	yes	—	2.49	0.00	16	$8.42 \cdot 10^{-7}$	yes	—	0.11	0.00
10^3	6	17	$4.93 \cdot 10^{-9}$	yes	—	2.65	0.00	16	$8.12 \cdot 10^{-7}$	yes	—	0.74	0.00
Avg (successes)		14	$4.57 \cdot 10^{-9}$	—	—	2.41	0.00	15	$8.57 \cdot 10^{-7}$	—	—	0.31	0.00
10^4	1	8	$1.82 \cdot 10^{-12}$	yes	—	2.50	0.01	14	$8.38 \cdot 10^{-7}$	yes	—	0.17	0.03
10^4	2	22	$4.85 \cdot 10^{-7}$	yes	—	0.53	0.03	17	$8.78 \cdot 10^{-7}$	yes	—	0.42	0.03
10^4	3	20	$2.59 \cdot 10^{-7}$	yes	—	0.69	0.03	16	$8.21 \cdot 10^{-7}$	yes	—	0.69	0.03
10^4	4	20	$5.83 \cdot 10^{-7}$	yes	—	0.22	0.03	18	$7.14 \cdot 10^{-7}$	yes	—	0.22	0.04
10^4	5	22	$4.37 \cdot 10^{-7}$	yes	—	0.10	0.03	17	$9.38 \cdot 10^{-7}$	yes	—	0.45	0.04
10^4	6	19	$6.22 \cdot 10^{-7}$	yes	—	1.98	0.03	17	$9.71 \cdot 10^{-7}$	yes	—	0.45	0.04
Avg (successes)		19	$3.98 \cdot 10^{-7}$	—	—	1.00	0.03	17	$8.60 \cdot 10^{-7}$	—	—	0.40	0.04
10^5	1	8	$1.09 \cdot 10^{-13}$	yes	—	2.89	0.10	14	$9.96 \cdot 10^{-7}$	yes	—	0.53	0.41
10^5	2	50	$6.51 \cdot 10^{-5}$	no	maxit	—	0.60	20	$9.01 \cdot 10^{-7}$	yes	maxit	0.64	0.55
10^5	3	50	$3.80 \cdot 10^{-5}$	no	maxit	—	0.68	18	$8.86 \cdot 10^{-7}$	yes	—	0.16	0.45
10^5	4	50	$3.28 \cdot 10^{-5}$	no	maxit	—	0.62	17	$8.12 \cdot 10^{-7}$	yes	—	0.64	0.40
10^5	5	50	$7.29 \cdot 10^{-6}$	no	maxit	—	0.57	18	$9.77 \cdot 10^{-7}$	yes	—	0.40	0.46
10^5	6	50	$7.66 \cdot 10^{-5}$	no	maxit	—	0.57	17	$7.69 \cdot 10^{-7}$	yes	—	0.66	0.40
Avg (successes)		8	$1.09 \cdot 10^{-13}$	—	—	2.89	0.10	17	$8.90 \cdot 10^{-7}$	—	—	0.50	0.44

Table 3: Modified Newton vs Truncated Newton with exact derivatives on Problem 31 (Broyden31).

Dim	x_0	Modified Newton						Truncated Newton					
		iter	$\ \nabla f(x_k)\ $	success	flag	rate (exp.)	time (s)	iter	$\ \nabla f(x_k)\ $	success	flag	rate (exp.)	time (s)
2	1	5	$1.32 \cdot 10^{-7}$	yes	—	1.99	0.00	7	$6.17 \cdot 10^{-11}$	yes	—	1.99	0.00
2	2	8	$1.83 \cdot 10^{-13}$	yes	—	1.99	0.00	8	$9.93 \cdot 10^{-7}$	yes	—	1.97	0.00
2	3	5	$1.69 \cdot 10^{-7}$	yes	—	1.98	0.00	7	$1.30 \cdot 10^{-9}$	yes	—	1.99	0.00
2	4	7	$6.27 \cdot 10^{-7}$	yes	—	1.98	0.00	8	$3.99 \cdot 10^{-8}$	yes	—	1.99	0.00
2	5	6	$8.25 \cdot 10^{-11}$	yes	—	2.00	0.00	7	$9.30 \cdot 10^{-8}$	yes	—	1.99	0.00
2	6	6	$5.34 \cdot 10^{-8}$	yes	—	1.99	0.00	8	$1.69 \cdot 10^{-10}$	yes	—	1.99	0.00
Avg (successes)		6	$3.94 \cdot 10^{-8}$	—	—	1.99	0.00	8	$1.88 \cdot 10^{-7}$	—	—	1.99	0.00
10^3	1	5	$1.79 \cdot 10^{-8}$	yes	—	1.76	0.00	14	$7.58 \cdot 10^{-7}$	yes	—	0.23	0.00
10^3	2	10	$8.97 \cdot 10^{-8}$	yes	—	1.99	0.01	15	$8.55 \cdot 10^{-7}$	yes	—	0.35	0.00
10^3	3	11	$2.75 \cdot 10^{-9}$	yes	—	2.00	0.01	15	$9.17 \cdot 10^{-7}$	yes	—	0.24	0.00
10^3	4	9	$1.97 \cdot 10^{-12}$	yes	—	1.99	0.00	15	$9.61 \cdot 10^{-7}$	yes	—	0.16	0.00
10^3	5	10	$3.10 \cdot 10^{-12}$	yes	—	2.00	0.00	16	$8.42 \cdot 10^{-7}$	yes	—	0.11	0.00
10^3	6	9	$2.97 \cdot 10^{-7}$	yes	—	1.94	0.00	16	$8.12 \cdot 10^{-7}$	yes	—	0.74	0.00
Avg (successes)		9	$5.43 \cdot 10^{-9}$	—	—	1.95	0.00	15	$8.57 \cdot 10^{-7}$	—	—	0.31	0.00
10^4	1	5	$1.79 \cdot 10^{-8}$	yes	—	1.56	0.02	14	$8.38 \cdot 10^{-7}$	yes	—	0.17	0.03
10^4	2	11	$1.56 \cdot 10^{-8}$	yes	—	1.99	0.05	17	$8.78 \cdot 10^{-7}$	yes	—	0.42	0.03
10^4	3	10	$5.76 \cdot 10^{-7}$	yes	—	1.96	0.04	16	$8.21 \cdot 10^{-7}$	yes	—	0.69	0.03
10^4	4	11	$1.09 \cdot 10^{-11}$	yes	—	2.00	0.04	18	$7.14 \cdot 10^{-7}$	yes	—	0.22	0.04
10^4	5	10	$3.62 \cdot 10^{-10}$	yes	—	2.00	0.04	17	$9.38 \cdot 10^{-7}$	yes	—	0.45	0.04
10^4	6	11	$4.80 \cdot 10^{-8}$	yes	—	1.99	0.05	17	$9.71 \cdot 10^{-7}$	yes	—	0.45	0.04
Avg (successes)		10	$3.59 \cdot 10^{-8}$	—	—	1.92	0.04	17	$8.60 \cdot 10^{-7}$	—	—	0.40	0.04
10^5	1	5	$1.79 \cdot 10^{-8}$	yes	—	1.56	0.20	14	$9.96 \cdot 10^{-7}$	yes	—	0.53	0.41
10^5	2	9	$8.76 \cdot 10^{-10}$	yes	—	2.00	0.54	20	$9.01 \cdot 10^{-7}$	yes	maxit	0.64	0.55
10^5	3	9	$4.73 \cdot 10^{-13}$	yes	—	1.98	0.53	18	$8.86 \cdot 10^{-7}$	yes	—	0.16	0.45
10^5	4	12	$4.41 \cdot 10^{-13}$	yes	—	2.00	0.76	17	$8.12 \cdot 10^{-7}$	yes	—	0.64	0.40
10^5	5	12	$1.37 \cdot 10^{-11}$	yes	—	2.00	0.62	18	$9.77 \cdot 10^{-7}$	yes	—	0.40	0.46
10^5	6	11	$2.45 \cdot 10^{-9}$	yes	—	2.00	0.73	17	$7.69 \cdot 10^{-7}$	yes	—	0.66	0.40
Avg (successes)		10	$3.54 \cdot 10^{-9}$	—	—	1.92	0.56	17	$8.90 \cdot 10^{-7}$	—	—	0.50	0.44

Table 4: Trig16 – Tab k8_hi: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	5	$2.33e-12$	yes	–	–	0.00	5	$9.25e-13$	yes	–	–	0.00
2	rand1	4	$3.52e-13$	yes	–	2.921	0.00	5	$5.38e-14$	yes	–	2.984	0.00
2	rand2	5	$5.74e-10$	yes	–	–	0.00	5	$3.14e-14$	yes	–	2.236	0.00
2	rand3	5	$1.07e-12$	yes	–	2.988	0.00	5	$2.67e-12$	yes	–	2.742	0.00
2	rand4	4	$1.21e-14$	yes	–	2.082	0.00	5	$3.16e-07$	yes	–	–	0.00
2	rand5	3	$2.41e-07$	yes	–	2.914	0.00	6	$2.88e-11$	yes	–	2.964	0.00
10^3	xbar	8	$1.07e-11$	yes	–	2.968	0.02	22	$7.36e-08$	yes	–	1.491	0.07
10^3	rand1	15	$6.75e-09$	yes	–	1.348	0.03	27	$5.71e-08$	yes	–	1.294	0.09
10^3	rand2	15	$1.80e-07$	yes	–	–	0.03	23	$5.31e-07$	yes	–	1.550	0.07
10^3	rand3	16	$5.44e-07$	yes	–	–	0.03	25	$5.94e-08$	yes	–	0.625	0.07
10^3	rand4	16	$4.73e-09$	yes	–	2.190	0.03	25	$6.50e-07$	yes	–	1.512	0.07
10^3	rand5	17	$2.74e-09$	yes	–	1.352	0.03	25	$8.49e-08$	yes	–	1.538	0.07
10^4	xbar	8	$1.08e-11$	yes	–	2.966	0.15	26	$3.75e-07$	yes	–	1.000	0.67
10^4	rand1	18	$4.70e-07$	yes	–	–	0.30	39	$5.04e-08$	yes	–	1.512	1.00
10^4	rand2	21	$5.21e-07$	yes	–	0.537	0.33	30	$1.90e-08$	yes	–	1.018	0.77
10^4	rand3	24	$2.77e-07$	yes	–	2.276	0.39	33	$4.90e-07$	yes	–	1.494	0.84
10^4	rand4	21	$1.66e-07$	yes	–	0.558	0.33	37	$1.84e-08$	yes	–	1.324	0.94
10^4	rand5	19	$1.43e-07$	yes	–	0.901	0.31	37	$2.08e-06$	no	maxit	0.000	0.98
10^5	xbar	8	$1.07e-11$	yes	–	2.968	1.49	29	$9.18e-08$	yes	–	1.505	9.04
10^5	rand1	50	$8.24e-05$	no	maxit	–	7.38	46	$6.80e-07$	yes	–	1.469	17.00
10^5	rand2	50	$9.17e-05$	no	maxit	–	7.38	16	$1.71e+05$	no	bt/maxit	0.000	4.89
10^5	rand3	50	$8.15e-05$	no	maxit	–	7.39	17	$9.79e+04$	no	bt/maxit	0.000	4.21
10^5	rand4	50	$5.47e-05$	no	maxit	–	7.38	43	$1.25e-05$	no	maxit	0.000	13.73
10^5	rand5	9	$2.60e+05$	no	bt/maxit	0.000	1.45	43	$8.22e-06$	no	maxit	0.000	13.24

Table 5: Broyden31 – Tab k8_hi: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	5	$1.32e-07$	yes	–	1.990	0.00	7	$1.32e-07$	yes	–	1.991	0.00
2	rand1	6	$7.03e-09$	yes	–	1.987	0.00	8	$7.04e-09$	yes	–	1.987	0.00
2	rand2	6	$9.52e-13$	yes	–	2.004	0.00	7	$1.03e-12$	yes	–	1.993	0.00
2	rand3	6	$5.77e-08$	yes	–	1.991	0.00	8	$5.78e-08$	yes	–	1.991	0.00
2	rand4	7	$3.51e-07$	yes	–	1.981	0.00	7	$3.51e-07$	yes	–	1.981	0.00
2	rand5	7	$3.61e-13$	yes	–	2.018	0.00	6	$4.23e-13$	yes	–	1.997	0.00
10^3	xbar	5	$1.79e-08$	yes	–	1.760	0.02	5	$1.79e-08$	yes	–	1.760	0.00
10^3	rand1	9	$6.42e-12$	yes	–	1.996	0.03	9	$6.60e-12$	yes	–	1.992	0.00
10^3	rand2	9	$4.91e-07$	yes	–	1.978	0.03	9	$4.91e-07$	yes	–	1.978	0.00
10^3	rand3	9	$8.80e-07$	yes	–	1.982	0.03	9	$8.80e-07$	yes	–	1.982	0.01
10^3	rand4	9	$1.38e-10$	yes	–	1.971	0.03	9	$1.38e-10$	yes	–	1.970	0.00
10^3	rand5	2	$8.45e+02$	no	bt/maxit	–	0.01	2	$8.45e+02$	no	bt/maxit	–	0.00
10^4	xbar	5	$1.79e-08$	yes	–	1.559	0.17	5	$1.79e-08$	yes	–	1.559	0.02
10^4	rand1	10	$2.50e-10$	yes	–	2.000	0.34	10	$2.50e-10$	yes	–	2.000	0.04
10^4	rand2	11	$2.47e-07$	yes	–	1.990	0.40	11	$2.47e-07$	yes	–	1.990	0.05
10^4	rand3	11	$5.14e-13$	yes	–	2.013	0.36	11	$5.72e-13$	yes	–	1.999	0.05
10^4	rand4	10	$1.53e-12$	yes	–	2.004	0.34	10	$1.60e-12$	yes	–	1.999	0.05
10^4	rand5	10	$1.25e-10$	yes	–	1.999	0.34	10	$1.26e-10$	yes	–	1.999	0.04
10^5	xbar	5	$1.79e-08$	yes	–	1.555	2.04	5	$1.79e-08$	yes	–	1.555	0.27
10^5	rand1	12	$6.76e-10$	yes	–	1.983	4.16	12	$6.76e-10$	yes	–	1.983	0.79
10^5	rand2	12	$1.83e-13$	yes	–	2.021	3.93	12	$2.18e-13$	yes	–	1.999	0.65
10^5	rand3	12	$1.48e-10$	yes	–	2.002	4.32	12	$1.48e-10$	yes	–	2.001	0.70
10^5	rand4	12	$1.66e-10$	yes	–	2.000	4.07	12	$1.67e-10$	yes	–	1.999	0.66
10^5	rand5	10	$2.15e-10$	yes	–	2.001	3.67	10	$2.15e-10$	yes	–	2.000	0.77

Table 6: Trig16 – k8_h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	3	$2.48e+00$	no	bt/maxit	0.000	0.00	3	$3.27e+00$	no	maxit	0.000	0.00
2	rand1	2	$2.85e+00$	no	bt/maxit	0.000	0.00	3	$2.59e+00$	no	maxit	0.000	0.00
2	rand2	4	$3.25e+00$	no	bt/maxit	0.000	0.00	4	$2.49e+00$	no	maxit	0.000	0.00
2	rand3	3	$2.73e+00$	no	bt/maxit	0.000	0.00	2	$2.51e+00$	no	maxit	–	0.00
2	rand4	1	$4.63e-01$	no	bt/maxit	0.000	0.00	2	$4.63e-01$	no	maxit	–	0.00
2	rand5	6	$2.52e+00$	no	bt/maxit	0.000	0.00	3	$5.04e-02$	no	maxit	0.000	0.00
10^3	xbar	6	$6.43e+03$	no	bt/maxit	0.000	0.00	18	$2.88e+03$	no	maxit	0.000	0.01
10^3	rand1	20	$4.45e+03$	no	bt/maxit	0.000	0.01	21	$7.97e+03$	no	maxit	0.000	0.01
10^3	rand2	15	$6.72e+03$	no	bt/maxit	0.000	0.01	31	$8.71e+03$	no	maxit	0.000	0.02
10^3	rand3	9	$1.16e+04$	no	bt/maxit	0.000	0.01	17	$7.35e+03$	no	maxit	0.000	0.01
10^3	rand4	31	$2.71e+03$	no	bt/maxit	0.000	0.02	51	$7.81e+03$	no	maxit	–	0.03
10^3	rand5	19	$5.11e+03$	no	bt/maxit	0.000	0.01	51	$7.82e+03$	no	maxit	0.237	0.03
10^4	xbar	1	$4.86e+05$	no	bt/maxit	0.000	0.01	2	$4.86e+05$	no	maxit	–	0.12
10^4	rand1	23	$9.96e+04$	no	bt/maxit	0.000	0.06	51	$2.40e+05$	no	maxit	–	0.16
10^4	rand2	20	$2.32e+05$	no	bt/maxit	0.000	0.07	44	$3.00e+05$	no	maxit	0.000	0.15
10^4	rand3	25	$9.58e+04$	no	bt/maxit	0.000	0.08	51	$2.85e+05$	no	maxit	–	0.17
10^4	rand4	17	$2.37e+05$	no	bt/maxit	0.000	0.05	30	$3.06e+05$	no	maxit	0.000	0.08
10^4	rand5	19	$2.39e+05$	no	bt/maxit	0.000	0.06	39	$2.88e+05$	no	maxit	0.000	0.11
10^5	xbar	6	$7.27e+06$	no	bt/maxit	0.000	0.27	51	$4.59e+06$	no	maxit	–	0.89
10^5	rand1	8	$1.49e+07$	no	bt/maxit	0.000	0.29	51	$8.18e+06$	no	maxit	0.746	1.00
10^5	rand2	21	$9.55e+06$	no	bt/maxit	0.000	0.87	51	$7.45e+06$	no	maxit	–	0.98
10^5	rand3	21	$6.51e+06$	no	bt/maxit	0.000	0.53	51	$8.10e+06$	no	maxit	2.425	0.96
10^5	rand4	5	$1.63e+07$	no	bt/maxit	0.000	0.21	51	$7.94e+06$	no	maxit	0.388	0.98
10^5	rand5	24	$4.32e+06$	no	bt/maxit	0.000	0.64	51	$7.18e+06$	no	maxit	1.103	0.88

Table 7: Broyden31 – Tab k8_h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	6	$3.51e-09$	yes	–	1.979	0.00	7	$6.17e-11$	yes	–	1.992	0.02
2	rand1	7	$1.73e-09$	yes	–	1.982	0.00	8	$5.60e-10$	yes	–	1.992	0.00
2	rand2	6	$3.12e-12$	yes	–	2.015	0.00	7	$1.15e-08$	yes	–	1.991	0.00
2	rand3	7	$3.48e-13$	yes	–	1.987	0.00	8	$4.88e-10$	yes	–	1.992	0.00
2	rand4	6	$1.33e-07$	yes	–	2.011	0.00	7	$8.01e-08$	yes	–	1.987	0.00
2	rand5	7	$2.43e-10$	yes	–	2.005	0.00	6	$4.72e-08$	yes	–	1.981	0.00
10^3	xbar	13	$7.22e-07$	yes	–	0.815	0.05	13	$1.91e-07$	yes	–	1.185	0.01
10^3	rand1	14	$3.08e-07$	yes	–	1.050	0.06	15	$1.56e-07$	yes	–	0.879	0.00
10^3	rand2	14	$5.53e-07$	yes	–	1.015	0.07	15	$6.72e-07$	yes	–	0.820	0.00
10^3	rand3	14	$6.19e-07$	yes	–	0.860	0.07	15	$2.27e-07$	yes	–	1.016	0.00
10^3	rand4	14	$5.33e-07$	yes	–	0.877	0.07	15	$1.44e-07$	yes	–	1.154	0.00
10^3	rand5	15	$8.56e-08$	yes	–	0.887	0.07	15	$1.76e-07$	yes	–	0.952	0.00
10^4	xbar	13	$5.10e-07$	yes	–	1.036	1.55	13	$6.24e-07$	yes	–	0.793	0.03
10^4	rand1	16	$3.07e-07$	yes	–	0.942	1.89	16	$6.68e-07$	yes	–	0.936	0.03
10^4	rand2	16	$1.63e-07$	yes	–	0.893	1.87	16	$3.95e-07$	yes	–	0.824	0.05
10^4	rand3	15	$3.13e-07$	yes	–	0.907	1.70	16	$1.32e-07$	yes	–	1.028	0.04
10^4	rand4	16	$7.18e-07$	yes	–	1.012	1.76	17	$5.02e-07$	yes	–	0.891	0.04
10^4	rand5	16	$3.78e-07$	yes	–	0.951	1.77	16	$6.56e-07$	yes	–	0.906	0.04
10^5	xbar	4	$3.37e-09$	yes	–	1.782	1.00	5	$1.32e-07$	yes	–	1.990	0.08
10^5	rand1	6	$1.06e-09$	yes	–	1.982	1.52	6	$7.04e-09$	yes	–	1.987	0.00
10^5	rand2	6	$1.81e-12$	yes	–	1.928	1.46	6	$1.03e-12$	yes	–	1.993	0.00
10^5	rand3	6	$1.40e-10$	yes	–	1.967	1.47	6	$5.78e-08$	yes	–	1.991	0.00
10^5	rand4	6	$7.23e-13$	yes	–	1.916	1.47	7	$3.51e-07$	yes	–	1.981	0.00
10^5	rand5	6	$1.47e-10$	yes	–	1.984	1.53	7	$4.23e-13$	yes	–	1.997	0.00

A Code

```
1 function [f,gradf,hessf,xbar,tfun] = problem_trig16()
2
3 f      = @Ffun;
4 gradf  = @gfun;
5 hessf  = @Hfun;
6 tfun   = @rvec;
7
8 xbar = @(n) ones(n,1);
9
10 function r = rvec(x)
11     x = x(:);
12     n = length(x);
13     i = (1:n)';
14
15     xm1 = [0; x(1:n-1)];
16     xp1 = [x(2:n); 0];
17
18     r = i .* ((1 - cos(x)) + sin(xm1) - sin(xp1));
19 end
20
21 function Fx = Ffun(x)
22     n = length(x);
23     i = (1:n)';
24     Fx = sum(i .* (1 - cos(x))) ...
25         + 2 * sum(sin(x(1:n-1))) ...
26         - (n-1) * sin(x(n));
27 end
28
29 function g = gfun(x)
30     n = length(x);
31     i = (1:n)';
32     g = zeros(n,1);
33     g(1:n-1) = i(1:n-1).*sin(x(1:n-1)) + 2*cos(x(1:n-1));
34     g(n)      = n*sin(x(n)) - (n-1)*cos(x(n));
35 end
36
37 function H = Hfun(x)
38     n = length(x);
39     i = (1:n)';
40     d = zeros(n,1);
41     d(1:n-1) = i(1:n-1).*cos(x(1:n-1)) - 2*sin(x(1:n-1));
42     d(n)      = n*cos(x(n)) + (n-1)*sin(x(n));
43     H = spdiags(d,0,n,n);
44 end
45 end
```

Listing 1: Problem 16: Banded Trigonometric

```
1 function [f,gradf,hessf,xbar,rfun] = problem_broyden31()
2
3 f      = @Ffun;
4 gradf  = @gfun;
5 hessf  = @Hfun;
6 rfun   = @rvec;
7
8 xbar = @(n) -ones(n,1);
9
10 function r = rvec(x)
11     n = length(x);
12     xm1 = [0; x(1:n-1)];
13     xp1 = [x(2:n); 0];
14     r = (3 - 2*x).*x - xm1 - 2*xp1 + 1;
15 end
16
```

```

17 function Fx = Ffun(x)
18     r = rvec(x);
19     Fx = 0.5 * (r' * r);
20 end
21
22 function g = gfun(x)
23     n = length(x);
24     r = rvec(x);
25     d = 3 - 4*x;
26     J = spdiags([-ones(n,1), d, -2*ones(n,1)],[-1,0,1],n,n);
27     g = J' * r;
28 end
29
30 function H = Hfun(x)
31     n = length(x);
32     r = rvec(x);
33     d = 3 - 4*x;
34     J = spdiags([-ones(n,1), d, -2*ones(n,1)],[-1,0,1],n,n);
35     H = J' * J - 4 * spdiags(r,0,n,n);
36 end
37 end

```

Listing 2: Problem 31: Broyden Tridiagonal Least-Squares

```

1 function [xk,fk,gradfk_norm,k,xseq,btseq,alphas,gradfk_seq,fk_seq,tau_new,pks] =
   modified_newton_method(x0,f,gradf,hessf,kmax,tolgrad,c1,rho,btmax,beta)
2 % MODIFIED_NEWTON_METHOD Modified Newton method with Hessian correction
3 %
4 % INPUT ARGUMENTS:
5 %     x0           : initial point (column vector)
6 %     f            : function handle for the scalar objective function f(x)
7 %     gradf        : function handle for the gradient gradf(x)
8 %     hessf        : function handle for the Hessian hessf(x)
9 %     kmax         : maximum number of iterations
10 %    tolgrad       : tolerance on the gradient norm for the stopping condition
11 %    c1, rho, btmax : parameters for Armijo/backtracking (0 < c1 < 1, 0 < rho < 1)
12 %    beta          : minimum initial increment for the Hessian correction (beta >
   0)
13
14 % INPUT CHECKS
15 if ~isnumeric(x0) || ~isvector(x0)
16     error('x0 must be a numeric column vector. ');
17 end
18 xk = x0(:);
19 n = length(xk);
20
21 if ~isa(f,'function_handle')
22     error('f must be a function handle. ');
23 end
24
25 try
26     fk = f(xk);
27 catch
28     error('f(x0) cannot be evaluated. ');
29 end
30
31 if ~isscalar(fk) || ~isreal(fk)
32     error('f(x) must return a real scalar. ');
33 end
34
35 if ~isa(gradf,'function_handle')
36     error('gradf must be a function handle. ');
37 end
38
39 try
40     gradfk = gradf(xk);
41 catch

```

```

42     error('gradf(x0) cannot be evaluated. ');
43 end
44
45 gradfk_norm = norm(gradfk);
46 farmijo = @(fk, alpha, c1_gradfk_pk) fk + alpha * c1_gradfk_pk;
47
48 % Inner parameters
49 tol = 1e-6;
50 maxit = 200;
51
52 % Preallocations
53 xseq = zeros(length(x0), kmax);
54 btseq = zeros(1, kmax);
55 alphas = zeros(1, kmax);
56 gradfk_seq = zeros(1, kmax);
57 fk_seq = zeros(1, kmax);
58 pks = zeros(length(x0), kmax);
59 tau_new = zeros(maxit+1, kmax);
60 k = 0;
61
62 while k < kmax && gradfk_norm >= tolgrad
63     Hk = hessf(xk);
64     diagHk = full(diag(Hk));
65     isPositive = all(diagHk>tol);
66
67     if isPositive == true
68         tau_k = 0;
69     else
70         tau_k = beta-min(diagHk);
71     end
72
73     tauk = zeros(maxit+1,1);
74     tauk(1)= tau_k;
75
76     for j = 1:maxit
77         if n > 1e-4
78             Bk = Hk+tauk(j)*speye(n);
79         else
80             Bk = Hk+tauk(j)*eye(n);
81         end
82
83         [R,flag] = chol(Bk);
84         if flag == 0
85             break
86         else
87             tauk(j+1) = max(beta, 2*tauk(j));
88         end
89     end
90
91     pk = R\' \ (-gradfk);
92
93     % BACKTRACKING
94     alpha = 1;
95     xnew = xk + alpha * pk;
96     fnew = f(xnew);
97     c1_gradfk_pk = c1 * gradfk' * pk;
98     bt = 0;
99
100    while bt < btmax && fnew > farmijo(fk, alpha, c1_gradfk_pk)
101        alpha = rho * alpha;
102        xnew = xk + alpha * pk;
103        fnew = f(xnew);
104        bt = bt + 1;
105    end
106
107    if bt == btmax && fnew > farmijo(fk, alpha, c1_gradfk_pk)

```



```

108     k = k+1;
109     xseq(:, k) = xk;
110     btseq(k) = bt;
111     tau_new(:, k) = tauk;
112     alphas(k) = alpha;
113     pks(:, k) = pk;
114     gradfk_seq(k) = gradfk_norm;
115     fk_seq(k) = fk;
116     break;
117 end
118
119 xk = xnew;
120 fk = fnew;
121 gradfk = gradf(xk);
122 gradfk_norm = norm(gradfk);
123
124 k = k + 1;
125 xseq(:, k) = xk;
126 btseq(k) = bt;
127 tau_new(:, k) = tauk;
128 alphas(k) = alpha;
129 pks(:, k) = pk;
130 gradfk_seq(k) = gradfk_norm;
131 fk_seq(k) = fk;
132 end
133
134 % Trimming
135 xseq = xseq(:, 1:k);
136 btseq = btseq(1:k);
137 alphas = alphas(1:k);
138 pks = pks(:, 1:k);
139 gradfk_seq = gradfk_seq(1:k);
140 fk_seq = fk_seq(1:k);
141 xseq = [x0, xseq];
142 end

```

Listing 3: Modified Newton method with Hessian regularization

```

1 function [xk,fk,gradfk_norm,k,xseq,btseq,pks,inner_iters] = truncated_newton_method(x0,
   f,gradf,hessf,kmax,tolgrad,c1,rho,btmax,max_cg)
2 % TRUNCATED-NEWTON-METHOD Truncated Newton method (Newton-CG)
3 %
4 % INPUT ARGUMENTS:
5 %     x0           : Initial guess (column vector)
6 %     f            : Function handle for the objective function f(x)
7 %     gradf        : Function handle for the gradient gradf(x)
8 %     hessf        : Function handle for the Hessian matrix H(x)
9 %     kmax         : Maximum number of outer iterations
10 %    tolgrad       : Stopping tolerance on the norm of the gradient
11 %     c1           : Armijo condition parameter (0 < c1 < 1)
12 %     rho          : Backtracking reduction factor (0 < rho < 1)
13 %     btmax        : Maximum number of backtracking steps per iteration
14 %     max_cg       : Maximum number of inner (CG) iterations
15
16 % INPUT CHECKS
17 if ~isnumeric(x0) || ~isvector(x0)
18     error('x0 must be a numeric column vector.');
```

```

19 end
20 xk = x0(:); % force column vector
21 n = length(xk);
22
23 if ~isa(f,'function_handle') || ~isa(gradf,'function_handle') || ~isa(hessf,'
   function_handle')
24     error('f, gradf and hessf must be function handles.');
```

```

25 end
26
27 fk = f(xk);

```

```

28 gradfk = gradf(xk);
29 gradfk_norm = norm(gradfk);
30
31 if gradfk_norm < tolgrad
32     warning('Initial point already satisfies the stopping criterion.');
```

33 end

34

35 % Function handle for the armijo condition.

36 farmijo = @(fk, alpha, c1_gradfk_pk) fk + alpha * c1_gradfk_pk;

37

38 % Preallocations.

39 xseq = zeros(length(x0), kmax);

40 btseq = zeros(1, kmax);

41 alphas = zeros(1, kmax);

42 pks = zeros(length(x0), kmax);

43 inner_iters = zeros(1, kmax);

44

45 k = 0;

46 while k < kmax && gradfk_norm >= tolgrad

47 % System solve: Hess(fk)*pk = -grad(fk)

48 z = zeros(length(x0),1);

49 Hk = hessf(xk);

50 ck = -gradfk;

51

52 p_tn = ck;

53 eta_k = min(0.5, sqrt(gradfk_norm));

54 r = ck - Hk * z;

55 r_old = r'*r;

56 dk = r;

57 stop_inner = false;

58

59 j = 0;

60 while ~stop_inner && j < max_cg

61 Hdk = Hk*dk;

62 curv = dk'*Hdk;

63

64 if curv <= 1e-10

65 if j == 0

66 p_tn = -gradfk;

67 else

68 p_tn = z;

69 end

70 stop_inner = true;

71 else

72 alpha_j = r_old/curv;

73 z = z + alpha_j * dk;

74 r = r - alpha_j * Hdk;

75

76 rk_new = r'*r;

77 beta_j = rk_new/r_old;

78 dk = r + beta_j * dk;

79 r_old = rk_new;

80 p_tn = z;

81 j = j+1;

82 end

83

84 if norm(r)/norm(ck) <= eta_k

85 p_tn = z;

86 stop_inner = true;

87 end

88 end

89

90 pk = p_tn;

91 inner_iters(k+1) = j;

92

93 % BACKTRACKING

```

94     alpha = 1;
95     xnew = xk + alpha * pk;
96     fnew = f(xnew);
97     c1_gradfk_pk = c1 * gradfk' * pk;
98     bt = 0;
99
100     while bt < btmax && fnew > farmijo(fk, alpha, c1_gradfk_pk)
101         alpha = rho * alpha;
102         xnew = xk + alpha * pk;
103         fnew = f(xnew);
104         bt = bt + 1;
105     end
106
107     if bt == btmax && fnew > farmijo(fk, alpha, c1_gradfk_pk)
108         disp('Backtracking: maximum iterations reached. ');
109         k = k+1;
110         xseq(:, k) = xk;
111         btseq(k) = bt;
112         pks(:, k) = pk;
113         break;
114     end
115
116     % Update variables
117     xk = xnew;
118     fk = fnew;
119     gradfk = gradf(xk);
120     gradfk_norm = norm(gradfk);
121     k = k + 1;
122
123     xseq(:, k) = xk;
124     btseq(k) = bt;
125     pks(:, k) = pk;
126 end
127
128 % Trimming
129 xseq = xseq(:, 1:k);
130 btseq = btseq(1:k);
131 pks = pks(:, 1:k);
132 xseq = [x0, xseq];
133 end

```

Listing 4: Truncated Newton method (Newton–CG)

```

1 function p = estimate_rate(xseq, gradf)
2     % Convergence exponent p estimation based on the last three gradient
3     % norms.
4     if isempty(xseq) || size(xseq,2) < 4
5         p = NaN; return;
6     end
7
8     g3 = norm(gradf(xseq(:, end)));
9     g2 = norm(gradf(xseq(:, end-1)));
10    g1 = norm(gradf(xseq(:, end-2)));
11
12    % Avoid division by zero or logarithms of extremely small numbers
13    if any([g1,g2,g3] < 1e-14)
14        p = NaN; return;
15    end
16
17    % Numerical estimation of the convergence rate
18    p = log(g3/g2) / log(g2/g1);
19
20    % Filter out non-significant or divergent values
21    if p < 0 || p > 3
22        p = NaN;
23    end
24 end

```

Listing 5: MATLAB function for estimating the convergence rate p .

```

1 function hvec = steps_fd(x, k, mode)
2 % This function computes the step size vector for finite difference approximations.
3 %
4 % INPUT ARGUMENTS:
5 %     x      : Current point (vector)
6 %     k      : Magnitude exponent (e.g., k=8 for 10^-8)
7 %     mode   : Selection mode:
8 %         'h' : Uniform step size h_i = 10^-k
9 %         'hi' : Relative step size h_i = 10^-k * max(|x_i|, 1)
10
11
12 x = x(:); % Ensure x is a column. vector
13 h = 10^(-k);
14
15 if strcmp(mode, 'hi')
16     % Compute step size relative to the magnitude of each component.
17     hvec = h * max(abs(x), 1);
18 else
19     % Compute a constant stepsize.
20     hvec = h * ones(size(x));
21 end
22
23 end

```

Listing 6: MATLAB function for computing finite difference step sizes.

```

1 function [g,H] = broyden_fd_grad_hess_GN(rfun, x, k, mode, bwJ)
2 % Estimating the Jacobian (J) of the residual r(x) using Central Finite Differences.
3 % 1. Sparsity: bwJ = 1 for Broyden31 (Tridiagonal structure).
4 % 2. Efficiency: 1D Coloring with stride = 2*bwJ + 1 (only 3 evaluations of r).
5 % 3. Computing:
6 %     Gradient: g = J' * r;
7 %     Hessian: H = J' * J.
8
9 x = x(:);
10 n = length(x);
11 % Calculate finite difference steps h.
12 h = steps_fd(x,k,mode);
13 r0 = rfun(x);
14
15 % Coloring Strategy to exploit sparsity.
16 stride = 2*bwJ + 1;
17 J = sparse(n,n);
18
19 for c = 1:stride
20     idx = c:stride:n;
21
22     dx = zeros(n,1);
23     dx(idx) = h(idx);
24
25     % Finite differences for the selected group.
26     rp = rfun(x + dx);
27     rm = rfun(x - dx);
28     dr = rp - rm;
29
30     % Unpack results into the sparse Jacobian matrix.
31     for jj = idx
32         rows = max(1, jj-bwJ):min(n, jj+bwJ);
33         J(rows, jj) = dr(rows) / (2*h(jj));
34     end
35 end
36
37 % Gradient: J' * r (Steepest descent direction)

```

```

38     g = J' * r0;
39
40     % Hessian Approximation: Gauss-Newton (J' * J)
41     % This is Symmetric Positive Semi-Definite (SPSD) by construction.
42     H = J' * J;
43     H = 0.5*(H + H'); % Enforce exact symmetry against numerical noise.
44 end

```

Listing 7: MATLAB implementation of broyden_fd_grad_hess_GN.

```

1 % This function executes the Modified and Truncated Newton method across multiple
2 % starting points to evaluate the robustness and efficiency of the
3 % provided oracles (Exact or FD).
4 % Output Data Structure
5 % res.fk:      Final objective function values for each start.
6 % res.gn:      Final gradient norms (used to check convergence).
7 % res.iters:   Number of iterations performed before stopping.
8 % res.time:    Wall-clock time spent on each optimization.
9 % res.success: Logical array (1 if ||g|| < tol, 0 otherwise).
10 function res = run_6starts_success(store, X0, f, gradf, hessf, kmax, tolgrad, c1, rho,
    btmax, max_cg)
11     num_pts = size(X0,2);
12
13     % Preallocations.
14     res = struct();
15     res.success = false(num_pts,1);
16     res.iters   = zeros(num_pts,1);
17     res.gn      = zeros(num_pts,1);
18     res.fk      = zeros(num_pts,1);
19     res.time    = zeros(num_pts,1);
20
21     for s = 1:num_pts
22         % Selecting starting points.
23         x0 = X0(:,s);
24
25         tic;
26         try
27             [~,fk, gn, it, xseq] = modified_newton_method(x0, f, gradf, hessf, ...
28                 %kmax, tolgrad, c1, rho, btmax, beta);
29             [~,fk, gn, it, xseq] = truncated_newton_method(x0, f, gradf, hessf, ...
30                 kmax, tolgrad, c1, rho, btmax, max_cg);
31         catch
32             % In case of failure.
33             fk = NaN; gn = Inf; it = 0;
34         end
35         t = toc;
36
37         rate = estimate_rate(xseq, gradf);
38         % Evaluate success based on the norm of the gradient.
39         succ = (gn < tolgrad);
40
41         % Storing.
42         res.fk(s)      = fk;
43         res.gn(s)      = gn;
44         res.iters(s)   = it;
45         res.time(s)    = t;
46         res.success(s) = succ;
47
48         startID = "xbar";
49         if s > 1, startID = "rand" + (s-1); end
50
51         fprintf("      %s: it=%d | ||g||=%.2e | f=%.3e | t=%.2fs | succ=%d | rate =%3
52             e \n", ...
53             startID, it, gn, fk, t, succ, rate);
54     end
55 end

```

Listing 8: MATLAB function to execute multiple optimization simulations.

```

1 % This script analyzes the performance of the Newton methods where the
2 % gradient is exact, while the Hessian is approximated via Finite Differences (FD).
3 % It evaluates the convergence and scalability (up to  $n=10^5$ ) across different
4 % FD step sizes ( $k=4, 8, 12$ ) and strategies.
5 clear; clc; close all;
6 addpath(genpath("C:\Users\giovannamaccarone\Desktop\Numerical04LSP\main"));
7 seed = 346710;
8 rng(seed,"twister");
9
10 % Working with two parameters, one fixed and the other depending on a k
11 % parameter.
12 n_list = [2, 1e3, 1e4, 1e5];
13 k_list = [4, 8, 12];
14 modes = ["h","hi"];
15
16 % Parameters (Modified Newton)
17 kmax = 50; % Max iterations.
18 tolgrad = 1e-6; % Tolerance for stopping criterion.
19 c1 = 1e-4; % Armijo sufficient decrease parameter.
20 rho = 0.3; % Backtracking reduction factor.
21 btmax = 10; % Max backtracking steps.
22 beta = 1e-2; % Hessian modification parameter.
23
24 % Problem definitions.
25 probs = {@problem_trig16};
26 pnames = "problem_trig16";
27
28 % Half-bandwidth for Hessian FD (0: diagonal, 1: tridiagonal, 2: pentadiagonal)
29 bwH = 0;
30
31 % Output initialization.
32 outdir = "out_fd_hess";
33 if ~exist(outdir,"dir"), mkdir(outdir); end
34 results = struct();
35
36 % Load problem data.
37 [f, grad_exact, hess_exact, xbarfun] = probs();
38 fprintf("FINITE DIFFERENCES HESS PROCESSING: %s\n", pname);
39 results.(pname) = struct();
40
41 for n = n_list
42     fprintf("\n--- n = %d ---\n", n);
43     % Generating starting points.
44     rng(seed + 1000*p + n, "twister");
45     xbar = xbarfun(n);
46     X0 = [xbar, xbar + (2*rand(n,5)-1)]; % 6 start: xbar + 5 random
47     dim_field = sprintf("n%d", n);
48     results.(pname).(dim_field) = struct();
49
50     % Only storing information about 2-dim.
51     store = (n == 2);
52
53     % CASE 1: Solving problem p with exact derivatives.
54     fprintf("CASE 1 (exact):\n");
55     res_exact = run_6starts_success(store, X0, f, grad_exact, hess_exact, ...
56         kmax, tolgrad, c1, rho, btmax, beta);
57     results.(pname).(dim_field).exact = res_exact;
58
59     % CASE 2: Solving problem p with exact gradient and approximate hessian.
60     for kk = 1:numel(k_list)
61         kfd = k_list(kk);
62         for mm = 1:numel(modes)
63             mode = char(modes(mm));

```

```

64         tag = sprintf("case1_k%d_%s", kfd, mode);
65         fprintf(" CASE 2 (approx hessian) %s:\n", tag);
66
67         % Hessian approximation via banded Finite Differences
68         Hfd = @(x) hess_fd_from_grad_banded(grad_exact, x, kfd, mode, bw);
69
70         res = run_6starts_success(store, X0, f, grad_exact, Hfd, ...
71             kmax, tolgrad, c1, rho, btmax, beta);
72         results.(pname).(dim_field).(tag) = res;
73     end
74 end
75 end
76
77 save(fullfile(outdir,"results_fd_hess.mat"),"results");
78 disp("Done. Saved in: " + fullfile(outdir,"results_fd_hess.mat"));

```

Listing 9: Simulation script for Newton methods with Finite Difference Hessian approximation.

```

1 % This script analyzes the performance of the Newton methods where both the
2 % gradient and the hessian are approximated via Finite Differences (FD).
3 % It evaluates the convergence and scalability (up to n=10^5) across different
4 % FD step sizes (k=4, 8, 12) and strategies.
5 clear; clc; close all;
6 addpath(genpath("C:\Users\giovannamaccarone\Desktop\Numerical04LSP\main"));
7 seed = 346710;
8 rng(seed,"twister");
9
10 % Working with two parameters, one fixed and the other depending on a k
11 % parameter.
12 n_list = [2, 1e3, 1e4, 1e5];
13 k_list = [4, 8, 12];
14 modes = ["h","hi"];
15
16 % Parameters (Modified Newton)
17 kmax = 50; % Max iterations.
18 tolgrad = 1e-6; % Tolerance for stopping criterion.
19 c1 = 1e-4; % Armijo sufficient decrease parameter.
20 rho = 0.3; % Backtracking reduction factor.
21 btmax = 10; % Max backtracking steps.
22 beta = 1e-2; % Hessian modification parameter.
23 % max_cg = 1000; % Max iterations for Conjugate Gradient.
24
25 % Problem definitions.
26 probs = {@problem_trig16};
27 pnames = "problem_trig16";
28
29 % Output initialization.
30 outdir = "out_fd_hess";
31 if ~exist(outdir,"dir"), mkdir(outdir); end
32 results = struct();
33
34 [f, grad_exact, hess_exact, xbarfun, rfun] = probs();
35 pname = pnames;
36 fprintf("FINITE DIFFERENCES HESS + GRAD PROCESSING: %s\n", pname);
37 results.(pname) = struct();
38
39 for n = n_list
40     fprintf("\n--- n = %d ---\n", n);
41     rng(seed + 1000*p + n, "twister");
42     xbar = xbarfun(n);
43     X0 = [xbar, xbar + (2*rand(n,5)-1)];
44     dim_field = sprintf("n%d", n);
45     results.(pname).(dim_field) = struct();
46     store = (n == 2);
47
48     % CASE 1: Solving problem p with exact derivatives.
49     fprintf("CASE 1 (exact):\n");

```

```

50     res_exact = run_simulation(store, X0, f, grad_exact, hess_exact, ...
51         kmax, tolgrad, c1, rho, btmax, max_cg);
52     results.(pname).(dim_field).exact = res_exact;
53
54     % CASE 2: Solving problem p approximating both gradient and hessian.
55     for kk = 1:numel(k_list) % Loop on possible k values.
56         kfd = k_list(kk);
57         for mm = 1:numel(modes) % Loop on possible ks (fixed or variable).
58             mode = char(modes(mm));
59             tag = sprintf("case2_k%d_%s", kfd, mode);
60             fprintf(" CASE 2 (approx grad + hess %s:\n", tag);
61
62             if pname == "problem_trig16"
63                 % Logic for Problem16:
64                 % Since Trig16 is a perfectly separable function, the Hessian is
65                 % diagonal.
66                 % We use optimized FD oracles that only compute n diagonal elements
67                 .
68                 tfun = rfun;
69                 grad_fd = @(x) trig_fd_grad_central_from_tfun(tfun, x, kfd, mode);
70                 hess_fd = @(x) trig_fd_hess_diag_central_from_tfun(tfun, x, kfd,
71                     mode);
72             else
73                 % Logic for Problem31:
74                 % Broyden31 has a tridiagonal Jacobian (bwJ = 1).
75                 % We approximate the gradient using a banded Jacobian (bwJ=1),
76                 % then the Hessian is computed via Gauss-Newton (J'J).
77                 bwJ = 1;
78                 [grad_fd, hess_fd] = broyden_fd_grad_hess_GN(rfun, kfd, mode, bwJ);
79             end
80
81             res = run_6starts_success(store, X0, f, grad_fd, hess_fd, ...
82                 kmax, tolgrad, c1, rho, btmax, max_cg);
83             results.(pname).(dim_field).(tag) = res;
84         end
85     end
86
87     save(fullfile(outdir, "results_fd_grad_hess.mat"), "results");
88     disp("Done. Saved in: " + fullfile(outdir, "results_fd_grad_hess.mat"));

```

Listing 10: Main simulation script for Newton methods with both Gradient and Hessian approximated via Finite Differences.

```

1 function H = trig_fd_hess_diag_central_from_tfun(tfun, x, k, mode)
2 % Diagonal Hessian estimation for the Trig16 problem using central finite differences.
3 % This approach exploits the separability of the objective function.
4
5 x = x(:);
6 n = length(x);
7
8 % Compute the optimal steps h.
9 h = steps_fd(x, k, mode);
10
11 % Evaluate the component-wise function at the center and perturbed points.
12 t0 = tfun(x);
13 tp = tfun(x + h);
14 tm = tfun(x - h);
15
16 % Apply the second-order central difference formula.
17 % Formula: H_ii approx (t_i(x_i + h) - 2*t_i(x_i) + t_i(x_i - h)) / h^2
18 d2 = (tp - 2*t0 + tm) ./ (h.^2);
19
20 % Construct the final sparse n-by-n diagonal matrix.
21 H = spdiags(d2, 0, n, n);
22 end

```


Listing 11: This function estimates diagonal Hessian using central finite differences for separable problems.

```
1 function g = trig_fd_grad_central_from_tfun(tfun, x, k, mode)
2 % Gradient estimation for the Trig16 problem via central finite differences.
3 % This function exploits the separability of the objective function.
4
5 x = x(:);
6
7 % Compute the optimal steps.
8 h = steps_fd(x, k, mode);
9
10 % Evaluate the component-wise function at perturbed points.
11 tp = tfun(x + h);
12 tm = tfun(x - h);
13
14 % Apply the first-order central difference formula for each component.
15 % Formula:  $g_i \approx (t_i(x_i + h) - t_i(x_i - h)) / (2h)$ 
16 g = (tp - tm) ./ (2*h);
17 end
```

Listing 12: This function estimates gradient using central finite differences for separable problems.

```
1 function H = hess_fd_from_grad_banded(gradf, x, k, mode, bw)
2 % Banded Hessian estimation via forward finite differences of the gradient.
3 % This function uses a 1D coloring strategy to compute the sparse Hessian
4 % matrix.
5 x = x(:);
6 n = length(x);
7
8 % Compute the optimal steps.
9 hvec = steps_fd(x, k, mode);
10
11 % Exact gradient evaluation at the current point.
12 g0 = gradf(x);
13
14 % Stride for the coloring strategy to avoid overlap within the band.
15 m = 2*bw + 1;
16
17 % Preallocation.
18 nnz_est = n * m;
19 I = zeros(nnz_est, 1);
20 J = zeros(nnz_est, 1);
21 V = zeros(nnz_est, 1);
22 ptr = 0;
23
24 for r = 1:m
25     % Identify indices of variables that can be perturbed simultaneously.
26     idx = r:m:n;
27
28     % Multi-directional perturbation vector.
29     e = zeros(n, 1);
30     e(idx) = hvec(idx);
31
32     % Perturbed gradient evaluation.
33     gr = gradf(x + e);
34
35     % Change in gradient.
36     dg = gr - g0;
37
38     for t = 1:numel(idx)
39         j = idx(t);
40         denom = hvec(j);
41
42         % Extract indices within the bandwidth bw for col j.
43         ii = (max(1, j-bw):min(n, j+bw))';
```

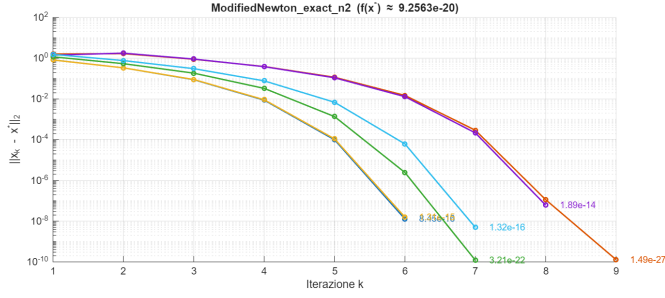
```

44
45     % Compute the j-th column entries.
46     ptr2 = ptr + numel(ii);
47     I(ptr+1:ptr2) = ii;
48     J(ptr+1:ptr2) = j;
49     V(ptr+1:ptr2) = dg(ii) / denom;
50     ptr = ptr2;
51 end
52 end
53
54 % Construct the sparse matrix from the accumulated triplets.
55 I = I(1:ptr); J = J(1:ptr); V = V(1:ptr);
56 H = sparse(I, J, V, n, n);
57
58 % Enforce symmetry to reduce numerical noise.
59 H = (H + H')/2;
60 end

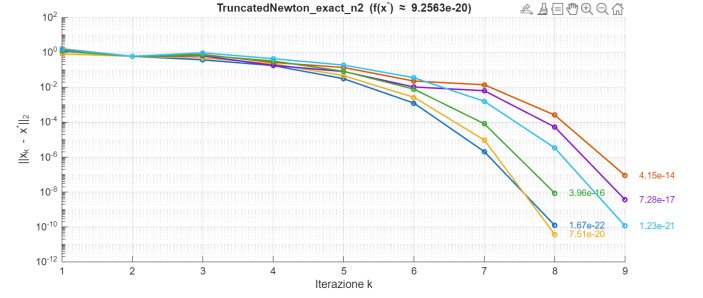
```

Listing 13: This function estimates banded Hessian via finite differences of the gradient using a coloring strategy.

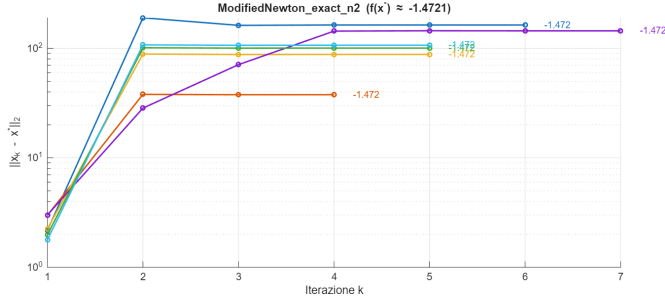
B Tables



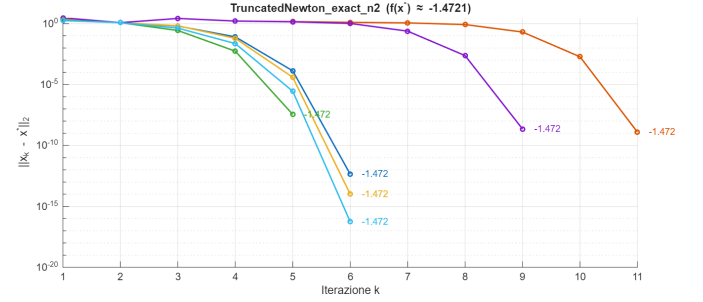
(a) Modified Exact on Broyden problem $N = 2$



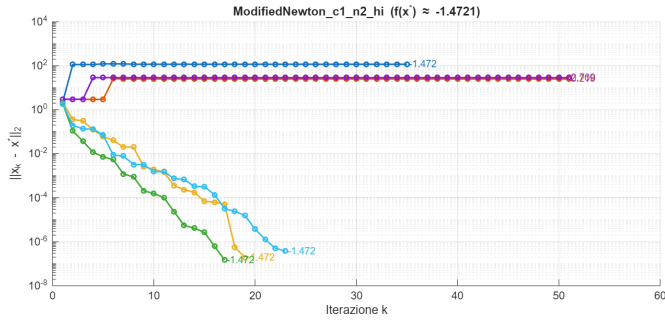
(b) Truncated Exact on Broyden problem $N = 2$



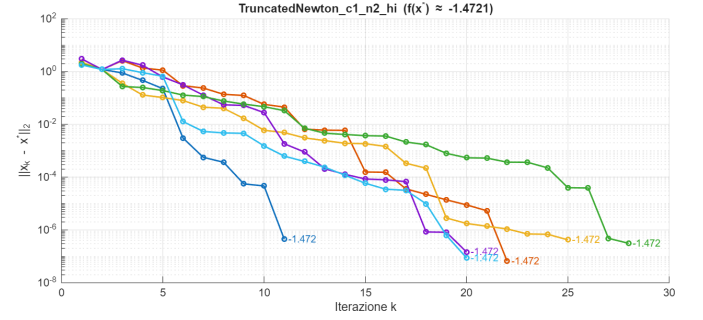
(c) Modified Exact on Trig16 problem $N = 2$



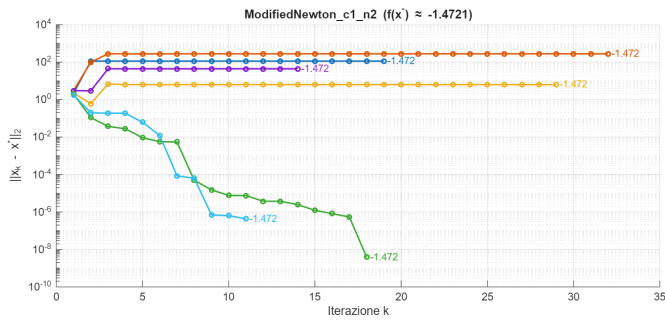
(d) Truncated Exact on Trig16 problem $N = 2$



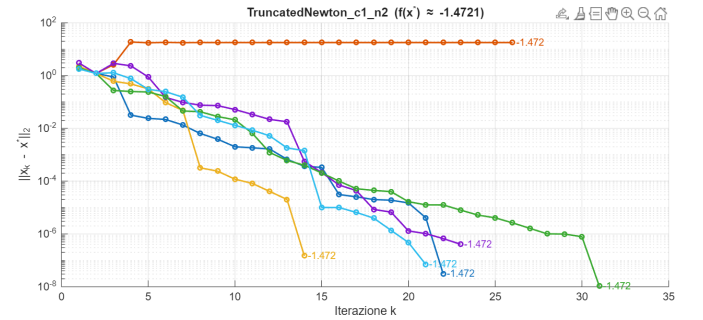
(e) Modified C1 $k = 8$, on Trig16 problem $N = 2$



(f) Truncated C1 $k = 8$, h_i mode on Trig16 problem $N = 2$



(g) Modified C1 $k = 8$, h_i mode on Trig16 problem $N = 2$



(h) Truncated C1 $k = 8$, on Trig16 problem $N = 2$

Figure 13: The first two plots show that, as the number of iterations increases, the computed solution approaches the theoretical minimizer, consistently with the local nature of both methods and with the convexity of the Broyden function. Plots (c)–(d) instead show that the algorithms converge to equivalent local minima, which may also be far from the theoretical minimum, in agreement with the periodic nature of the Trig16 function. These results highlight a strongly problem-dependent behavior. Finally, plots (e)–(f)–(g)–(h) show that the introduction of finite differences slows down the convergence of the methods, while adding noise that helps steering the algorithms toward the global minimum.

Table 8: Modified Newton vs Truncated Newton with exact derivatives on Problem 16 (Trig16).

Dim	x_0	Modified Newton						Truncated Newton					
		iter	$\ \nabla f(x_k)\ $	success	flag	rate conv.	time (s)	iter	$\ \nabla f(x_k)\ $	success	flag	rate (exp.)	time (s)
2	1	5	$3.23 \cdot 10^{-12}$	yes	—	2.99	0.00	7	$6.17 \cdot 10^{-11}$	yes	—	1.99	0.00
2	2	3	$5.63 \cdot 10^{-7}$	yes	—	2.91	0.00	8	$9.93 \cdot 10^{-7}$	yes	—	1.97	0.00
2	3	4	$1.02 \cdot 10^{-9}$	yes	—	NaN	0.00	7	$1.30 \cdot 10^{-9}$	yes	—	1.99	0.00
2	4	6	$1.88 \cdot 10^{-7}$	yes	—	NaN	0.00	8	$3.99 \cdot 10^{-8}$	yes	—	1.99	0.00
2	5	4	$4.20 \cdot 10^{-11}$	yes	—	NaN	0.00	7	$9.30 \cdot 10^{-8}$	yes	—	1.99	0.00
2	6	4	$1.21 \cdot 10^{-8}$	yes	—	NaN	0.00	8	$1.69 \cdot 10^{-10}$	yes	—	1.99	0.00
Avg (successes)		4	$1.27 \cdot 10^{-7}$	—	—	0.98	0.00	8	$1.88 \cdot 10^{-7}$	—	—	1.99	0.00
10^3	1	8	$1.53 \cdot 10^{-13}$	yes	—	2.84	0.00	14	$7.58 \cdot 10^{-7}$	yes	—	0.23	0.00
10^3	2	15	$4.79 \cdot 10^{-9}$	yes	—	2.03	0.00	15	$8.55 \cdot 10^{-7}$	yes	—	0.35	0.00
10^3	3	13	$6.92 \cdot 10^{-9}$	yes	—	2.33	0.00	15	$9.17 \cdot 10^{-7}$	yes	—	0.24	0.00
10^3	4	12	$7.95 \cdot 10^{-9}$	yes	—	2.11	0.00	15	$9.61 \cdot 10^{-7}$	yes	—	0.16	0.00
10^3	5	19	$2.85 \cdot 10^{-9}$	yes	—	2.49	0.00	16	$8.42 \cdot 10^{-7}$	yes	—	0.11	0.00
10^3	6	17	$4.93 \cdot 10^{-9}$	yes	—	2.65	0.00	16	$8.12 \cdot 10^{-7}$	yes	—	0.74	0.00
Avg (successes)		14	$4.57 \cdot 10^{-9}$	—	—	2.41	0.00	15	$8.57 \cdot 10^{-7}$	—	—	0.31	0.00
10^4	1	8	$1.82 \cdot 10^{-12}$	yes	—	2.50	0.01	14	$8.38 \cdot 10^{-7}$	yes	—	0.17	0.03
10^4	2	22	$4.85 \cdot 10^{-7}$	yes	—	0.53	0.03	17	$8.78 \cdot 10^{-7}$	yes	—	0.42	0.03
10^4	3	20	$2.59 \cdot 10^{-7}$	yes	—	0.69	0.03	16	$8.21 \cdot 10^{-7}$	yes	—	0.69	0.03
10^4	4	20	$5.83 \cdot 10^{-7}$	yes	—	0.22	0.03	18	$7.14 \cdot 10^{-7}$	yes	—	0.22	0.04
10^4	5	22	$4.37 \cdot 10^{-7}$	yes	—	0.10	0.03	17	$9.38 \cdot 10^{-7}$	yes	—	0.45	0.04
10^4	6	19	$6.22 \cdot 10^{-7}$	yes	—	1.98	0.03	17	$9.71 \cdot 10^{-7}$	yes	—	0.45	0.04
Avg (successes)		19	$3.98 \cdot 10^{-7}$	—	—	1.00	0.03	17	$8.60 \cdot 10^{-7}$	—	—	0.40	0.04
10^5	1	8	$1.09 \cdot 10^{-13}$	yes	—	2.89	0.10	14	$9.96 \cdot 10^{-7}$	yes	—	0.53	0.41
10^5	2	50	$6.51 \cdot 10^{-5}$	no	maxit	NaN	0.60	20	$9.01 \cdot 10^{-7}$	yes	maxit	0.64	0.55
10^5	3	50	$3.80 \cdot 10^{-5}$	no	maxit	NaN	0.68	18	$8.86 \cdot 10^{-7}$	yes	—	0.16	0.45
10^5	4	50	$3.28 \cdot 10^{-5}$	no	maxit	NaN	0.62	17	$8.12 \cdot 10^{-7}$	yes	—	0.64	0.40
10^5	5	50	$7.29 \cdot 10^{-6}$	no	maxit	NaN	0.57	18	$9.77 \cdot 10^{-7}$	yes	—	0.40	0.46
10^5	6	50	$7.66 \cdot 10^{-5}$	no	maxit	NaN	0.57	17	$7.69 \cdot 10^{-7}$	yes	—	0.66	0.40
Avg (successes)		8	$1.09 \cdot 10^{-13}$	—	—	2.89	0.10	17	$8.90 \cdot 10^{-7}$	—	—	0.50	0.44

Table 9: Modified Newton vs Truncated Newton with exact derivatives on Problem 31 (Broyden31).

Dim	x_0	Modified Newton						Truncated Newton					
		iter	$\ \nabla f(x_k)\ $	success	flag	rate (exp.)	time (s)	iter	$\ \nabla f(x_k)\ $	success	flag	rate (exp.)	time (s)
2	1	5	$1.32 \cdot 10^{-7}$	yes	—	1.99	0.00	7	$6.17 \cdot 10^{-11}$	yes	—	1.99	0.00
2	2	8	$1.83 \cdot 10^{-13}$	yes	—	1.99	0.00	8	$9.93 \cdot 10^{-7}$	yes	—	1.97	0.00
2	3	5	$1.69 \cdot 10^{-7}$	yes	—	1.98	0.00	7	$1.30 \cdot 10^{-9}$	yes	—	1.99	0.00
2	4	7	$6.27 \cdot 10^{-7}$	yes	—	1.98	0.00	8	$3.99 \cdot 10^{-8}$	yes	—	1.99	0.00
2	5	6	$8.25 \cdot 10^{-11}$	yes	—	2.00	0.00	7	$9.30 \cdot 10^{-8}$	yes	—	1.99	0.00
2	6	6	$5.34 \cdot 10^{-8}$	yes	—	1.99	0.00	8	$1.69 \cdot 10^{-10}$	yes	—	1.99	0.00
Avg (successes)		6	$3.94 \cdot 10^{-8}$	—	—	1.99	0.00	8	$1.88 \cdot 10^{-7}$	—	—	1.99	0.00
10^3	1	5	$1.79 \cdot 10^{-8}$	yes	—	1.76	0.00	14	$7.58 \cdot 10^{-7}$	yes	—	0.23	0.00
10^3	2	10	$8.97 \cdot 10^{-8}$	yes	—	1.99	0.01	15	$8.55 \cdot 10^{-7}$	yes	—	0.35	0.00
10^3	3	11	$2.75 \cdot 10^{-9}$	yes	—	2.00	0.01	15	$9.17 \cdot 10^{-7}$	yes	—	0.24	0.00
10^3	4	9	$1.97 \cdot 10^{-12}$	yes	—	1.99	0.00	15	$9.61 \cdot 10^{-7}$	yes	—	0.16	0.00
10^3	5	10	$3.10 \cdot 10^{-12}$	yes	—	2.00	0.00	16	$8.42 \cdot 10^{-7}$	yes	—	0.11	0.00
10^3	6	9	$2.97 \cdot 10^{-7}$	yes	—	1.94	0.00	16	$8.12 \cdot 10^{-7}$	yes	—	0.74	0.00
Avg (successes)		9	$5.43 \cdot 10^{-9}$	—	—	1.95	0.00	15	$8.57 \cdot 10^{-7}$	—	—	0.31	0.00
10^4	1	5	$1.79 \cdot 10^{-8}$	yes	—	1.56	0.02	14	$8.38 \cdot 10^{-7}$	yes	—	0.17	0.03
10^4	2	11	$1.56 \cdot 10^{-8}$	yes	—	1.99	0.05	17	$8.78 \cdot 10^{-7}$	yes	—	0.42	0.03
10^4	3	10	$5.76 \cdot 10^{-7}$	yes	—	1.96	0.04	16	$8.21 \cdot 10^{-7}$	yes	—	0.69	0.03
10^4	4	11	$1.09 \cdot 10^{-11}$	yes	—	2.00	0.04	18	$7.14 \cdot 10^{-7}$	yes	—	0.22	0.04
10^4	5	10	$3.62 \cdot 10^{-10}$	yes	—	2.00	0.04	17	$9.38 \cdot 10^{-7}$	yes	—	0.45	0.04
10^4	6	11	$4.80 \cdot 10^{-8}$	yes	—	1.99	0.05	17	$9.71 \cdot 10^{-7}$	yes	—	0.45	0.04
Avg (successes)		10	$3.59 \cdot 10^{-8}$	—	—	1.92	0.04	17	$8.60 \cdot 10^{-7}$	—	—	0.40	0.04
10^5	1	5	$1.79 \cdot 10^{-8}$	yes	—	1.56	0.20	14	$9.96 \cdot 10^{-7}$	yes	—	0.53	0.41
10^5	2	9	$8.76 \cdot 10^{-10}$	yes	—	2.00	0.54	20	$9.01 \cdot 10^{-7}$	yes	maxit	0.64	0.55
10^5	3	9	$4.73 \cdot 10^{-13}$	yes	—	1.98	0.53	18	$8.86 \cdot 10^{-7}$	yes	—	0.16	0.45
10^5	4	12	$4.41 \cdot 10^{-13}$	yes	—	2.00	0.76	17	$8.12 \cdot 10^{-7}$	yes	—	0.64	0.40
10^5	5	12	$1.37 \cdot 10^{-11}$	yes	—	2.00	0.62	18	$9.77 \cdot 10^{-7}$	yes	—	0.40	0.46
10^5	6	11	$2.45 \cdot 10^{-9}$	yes	—	2.00	0.73	17	$7.69 \cdot 10^{-7}$	yes	—	0.66	0.40
Avg (successes)		10	$3.54 \cdot 10^{-9}$	—	—	1.92	0.56	17	$8.90 \cdot 10^{-7}$	—	—	0.50	0.44

Table 10: Trig16 – Tab k4.h: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	5	$6.34e-12$	yes	–	2.885	0.01	5	$7.72e-13$	yes	–	NaN	0.01
2	rand1	4	$3.10e-14$	yes	–	NaN	0.00	5	$3.46e-14$	yes	–	NaN	0.00
2	rand2	5	$3.14e-10$	yes	–	2.964	0.00	5	$1.65e-14$	yes	–	2.336	0.00
2	rand3	5	$2.73e-12$	yes	–	2.842	0.00	5	$6.32e-14$	yes	–	NaN	0.00
2	rand4	4	$3.89e-15$	yes	–	NaN	0.00	5	$3.23e-07$	yes	–	NaN	0.00
2	rand5	3	$2.36e-07$	yes	–	2.920	0.00	6	$3.69e-11$	yes	–	2.920	0.00
10^3	xbar	6	$3.79e-07$	yes	–	2.730	0.01	20	$2.97e-07$	yes	–	1.518	0.08
10^3	rand1	14	$7.25e-09$	yes	–	1.972	0.02	25	$7.81e-07$	yes	–	1.487	0.09
10^3	rand2	13	$1.93e-08$	yes	–	2.878	0.02	24	$5.72e-08$	yes	–	0.664	0.07
10^3	rand3	13	$5.24e-08$	yes	–	2.540	0.02	24	$5.64e-07$	yes	–	0.223	0.08
10^3	rand4	16	$2.03e-09$	yes	–	2.936	0.04	27	$1.22e-07$	yes	–	1.522	0.08
10^3	rand5	14	$2.48e-09$	yes	–	2.216	0.03	27	$5.27e-08$	yes	–	0.823	0.10
10^4	xbar	6	$3.79e-07$	yes	–	1.525	0.13	26	$6.24e-08$	yes	–	NaN	0.75
10^4	rand1	18	$4.37e-07$	yes	–	2.079	0.37	38	$7.13e-08$	yes	–	1.517	0.97
10^4	rand2	21	$5.15e-07$	yes	–	0.709	0.52	33	$9.45e-06$	no	maxit	0.000	1.01
10^4	rand3	24	$2.50e-07$	yes	–	1.638	0.43	34	$9.84e-07$	yes	–	0.044	0.99
10^4	rand4	22	$2.21e-07$	yes	–	2.289	0.39	39	$5.26e-08$	yes	–	1.495	1.08
10^4	rand5	19	$1.01e-07$	yes	–	1.748	0.34	29	$5.09e-07$	yes	–	1.488	0.75
10^5	xbar	7	$2.49e-08$	yes	–	2.989	1.22	30	$2.63e-04$	no	maxit	0.000	8.36
10^5	rand1	50	$8.20e-05$	no	maxit	NaN	7.81	42	$2.11e-05$	no	maxit	0.000	11.85
10^5	rand2	50	$9.24e-05$	no	maxit	NaN	8.01	11	$4.12e+05$	no	maxit	0.000	2.70
10^5	rand3	50	$8.12e-05$	no	maxit	NaN	7.78	40	$8.14e-07$	yes	–	1.497	10.44
10^5	rand4	7	$1.14e+06$	no	bt/maxit	0.000	1.23	41	$1.48e-05$	no	maxit	0.000	11.19
10^5	rand5	8	$1.04e+06$	no	bt/maxit	0.000	1.36	51	$1.58e-06$	no	maxit	0.000	16.26

Table 11: Trig16 – Tab k4_hi: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	5	$3.21e-08$	yes	–	1.681	0.00	5	$8.48e-13$	yes	–	NaN	0.00
2	rand1	4	$1.05e-11$	yes	–	2.452	0.00	5	$5.92e-12$	yes	–	2.356	0.00
2	rand2	5	$3.06e-10$	yes	–	0.985	0.00	5	$8.51e-13$	yes	–	1.897	0.00
2	rand3	5	$8.15e-09$	yes	–	1.297	0.00	5	$6.00e-14$	yes	–	NaN	0.00
2	rand4	4	$2.51e-12$	yes	–	1.552	0.00	5	$3.23e-07$	yes	–	NaN	0.00
2	rand5	3	$4.60e-08$	yes	–	NaN	0.00	6	$3.77e-11$	yes	–	2.917	0.00
10^3	xbar	7	$1.66e-08$	yes	–	1.367	0.01	21	$6.54e-08$	yes	–	1.161	0.07
10^3	rand1	12	$2.52e+02$	no	bt/maxit	0.000	0.02	27	$8.89e-07$	yes	–	1.487	0.09
10^3	rand2	17	$5.20e+02$	no	bt/maxit	0.000	0.03	25	$2.25e-07$	yes	–	1.461	0.10
10^3	rand3	8	$9.82e+02$	no	bt/maxit	0.000	0.02	23	$1.52e-07$	yes	–	1.491	0.10
10^3	rand4	50	$6.85e-02$	no	maxit	NaN	0.08	27	$5.70e-08$	yes	–	1.115	0.08
10^3	rand5	7	$1.16e+03$	no	bt/maxit	0.000	0.01	24	$5.61e-08$	yes	–	1.169	0.08
10^4	xbar	7	$1.66e-08$	yes	–	1.367	0.11	27	$1.07e-05$	no	maxit	0.000	0.75
10^4	rand1	11	$6.99e+03$	no	bt/maxit	0.000	0.17	40	$8.17e-07$	yes	–	1.503	1.08
10^4	rand2	9	$1.13e+04$	no	bt/maxit	0.000	0.14	41	$1.14e-06$	no	maxit	0.000	1.12
10^4	rand3	13	$1.14e+04$	no	bt/maxit	0.000	0.20	35	$7.96e-08$	yes	–	1.497	0.96
10^4	rand4	7	$1.32e+04$	no	bt/maxit	0.000	0.11	41	$3.95e-06$	no	maxit	0.000	1.19
10^4	rand5	15	$5.14e+03$	no	bt/maxit	0.000	0.23	37	$8.12e-08$	yes	–	1.505	1.03
10^5	xbar	7	$1.66e-08$	yes	–	1.367	1.10	29	$5.52e-08$	yes	–	1.508	8.21
10^5	rand1	15	$4.33e+04$	no	bt/maxit	0.000	2.37	16	$1.64e+05$	no	bt/maxit	0.000	4.01
10^5	rand2	7	$1.50e+06$	no	bt/maxit	0.000	1.15	43	$4.75e-05$	no	maxit	0.000	13.03
10^5	rand3	8	$1.70e+05$	no	bt/maxit	0.000	1.25	41	$4.60e-06$	no	maxit	0.000	12.32
10^5	rand4	24	$1.24e+04$	no	bt/maxit	0.000	3.67	42	$1.79e-07$	yes	–	1.504	11.50
10^5	rand5	8	$2.27e+05$	no	bt/maxit	0.000	1.36	41	$1.30e-05$	no	maxit	0.000	12.20

Table 12: Trig16 – Tab k8.h: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	5	$2.78e-10$	yes	–	2.280	0.00	5	$7.92e-13$	yes	–	NaN	0.00
2	rand1	4	$1.49e-12$	yes	–	2.708	0.00	5	$8.17e-14$	yes	–	2.925	0.00
2	rand2	5	$1.05e-09$	yes	–	2.891	0.00	5	$6.01e-14$	yes	–	2.161	0.00
2	rand3	5	$1.94e-10$	yes	–	2.181	0.00	5	$1.87e-12$	yes	–	2.796	0.00
2	rand4	4	$2.58e-13$	yes	–	1.753	0.00	5	$3.17e-07$	yes	–	NaN	0.00
2	rand5	3	$2.40e-07$	yes	–	2.915	0.00	6	$2.45e-11$	yes	–	2.992	0.00
10^3	xbar	10	$5.19e-12$	yes	–	1.558	0.02	20	$2.74e-07$	yes	–	1.472	0.08
10^3	rand1	15	$6.27e-09$	yes	–	1.837	0.03	27	$8.78e-07$	yes	–	1.475	0.13
10^3	rand2	18	$3.31e-08$	yes	–	2.971	0.03	23	$5.32e-07$	yes	–	1.549	0.09
10^3	rand3	13	$2.62e-08$	yes	–	2.886	0.03	25	$5.71e-08$	yes	–	0.646	0.08
10^3	rand4	14	$5.23e-09$	yes	–	2.385	0.03	25	$6.49e-07$	yes	–	1.512	0.08
10^3	rand5	14	$2.15e-09$	yes	–	1.655	0.03	25	$1.11e-07$	yes	–	1.493	0.08
10^4	xbar	10	$5.50e-12$	yes	–	1.551	0.18	27	$9.63e-07$	yes	–	0.056	0.76
10^4	rand1	19	$2.03e-07$	yes	–	2.011	0.32	38	$1.91e-08$	yes	–	1.029	0.98
10^4	rand2	17	$5.12e-07$	yes	–	1.088	0.31	30	$4.94e-08$	yes	–	1.516	0.77
10^4	rand3	20	$4.27e-07$	yes	–	0.770	0.38	33	$8.62e-07$	yes	–	1.499	0.84
10^4	rand4	23	$2.69e-07$	yes	–	1.620	0.42	40	$1.65e-08$	yes	–	1.257	1.03
10^4	rand5	23	$1.33e-07$	yes	–	2.445	0.41	42	$3.48e-06$	no	maxit	0.000	1.34
10^5	xbar	10	$5.19e-12$	yes	–	1.558	1.88	32	$6.88e-06$	no	maxit	0.000	10.15
10^5	rand1	9	$3.40e+05$	no	bt/maxit	0.000	1.59	44	$6.77e-07$	yes	–	1.485	12.08
10^5	rand2	50	$9.54e-05$	no	maxit	NaN	7.73	43	$1.22e-07$	yes	–	1.499	11.60
10^5	rand3	10	$1.43e+05$	no	bt/maxit	0.000	1.61	17	$1.96e+05$	no	bt/maxit	0.000	4.24
10^5	rand4	31	$6.05e-05$	no	bt/maxit	0.000	4.97	40	$2.39e-07$	yes	–	0.352	11.04
10^5	rand5	50	$1.20e-05$	no	maxit	NaN	8.04	40	$2.97e-07$	yes	–	1.506	11.02

Table 13: Trig16 – Tab k8_hi: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	5	2.33e-12	yes	–	NaN	0.00	5	9.25e-13	yes	–	NaN	0.00
2	rand1	4	3.52e-13	yes	–	2.921	0.00	5	5.38e-14	yes	–	2.984	0.00
2	rand2	5	5.74e-10	yes	–	NaN	0.00	5	3.14e-14	yes	–	2.236	0.00
2	rand3	5	1.07e-12	yes	–	2.988	0.00	5	2.67e-12	yes	–	2.742	0.00
2	rand4	4	1.21e-14	yes	–	2.082	0.00	5	3.16e-07	yes	–	NaN	0.00
2	rand5	3	2.41e-07	yes	–	2.914	0.00	6	2.88e-11	yes	–	2.964	0.00
10 ³	xbar	8	1.07e-11	yes	–	2.968	0.02	22	7.36e-08	yes	–	1.491	0.07
10 ³	rand1	15	6.75e-09	yes	–	1.348	0.03	27	5.71e-08	yes	–	1.294	0.09
10 ³	rand2	15	1.80e-07	yes	–	NaN	0.03	23	5.31e-07	yes	–	1.550	0.07
10 ³	rand3	16	5.44e-07	yes	–	NaN	0.03	25	5.94e-08	yes	–	0.625	0.07
10 ³	rand4	16	4.73e-09	yes	–	2.190	0.03	25	6.50e-07	yes	–	1.512	0.07
10 ³	rand5	17	2.74e-09	yes	–	1.352	0.03	25	8.49e-08	yes	–	1.538	0.07
10 ⁴	xbar	8	1.08e-11	yes	–	2.966	0.15	26	3.75e-07	yes	–	1.000	0.67
10 ⁴	rand1	18	4.70e-07	yes	–	NaN	0.30	39	5.04e-08	yes	–	1.512	1.00
10 ⁴	rand2	21	5.21e-07	yes	–	0.537	0.33	30	1.90e-08	yes	–	1.018	0.77
10 ⁴	rand3	24	2.77e-07	yes	–	2.276	0.39	33	4.90e-07	yes	–	1.494	0.84
10 ⁴	rand4	21	1.66e-07	yes	–	0.558	0.33	37	1.84e-08	yes	–	1.324	0.94
10 ⁴	rand5	19	1.43e-07	yes	–	0.901	0.31	37	2.08e-06	no	maxit	0.000	0.98
10 ⁵	xbar	8	1.07e-11	yes	–	2.968	1.49	29	9.18e-08	yes	–	1.505	9.04
10 ⁵	rand1	50	8.24e-05	no	maxit	NaN	7.38	46	6.80e-07	yes	–	1.469	17.00
10 ⁵	rand2	50	9.17e-05	no	maxit	NaN	7.38	16	1.71e+05	no	bt/maxit	0.000	4.89
10 ⁵	rand3	50	8.15e-05	no	maxit	NaN	7.39	17	9.79e+04	no	bt/maxit	0.000	4.21
10 ⁵	rand4	50	5.47e-05	no	maxit	NaN	7.38	43	1.25e-05	no	maxit	0.000	13.73
10 ⁵	rand5	9	2.60e+05	no	bt/maxit	0.000	1.45	43	8.22e-06	no	maxit	0.000	13.24

Table 14: Trig16 – Tab k12_h: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	6	3.98e-08	yes	–	0.995	0.00	5	1.41e-08	yes	–	1.555	0.00
2	rand1	4	1.64e-08	yes	–	1.328	0.00	5	5.27e-09	yes	–	1.392	0.00
2	rand2	6	6.42e-07	yes	–	0.998	0.00	5	7.08e-10	yes	–	0.983	0.00
2	rand3	6	3.36e-08	yes	–	0.998	0.00	5	1.05e-08	yes	–	1.446	0.00
2	rand4	4	1.17e-07	yes	–	1.004	0.00	5	4.64e-07	yes	–	2.916	0.00
2	rand5	4	4.62e-08	yes	–	0.993	0.00	6	3.41e-08	yes	–	1.739	0.00
10 ³	xbar	7	6.90e-08	yes	–	0.992	0.01	20	1.26e-07	yes	–	1.504	0.06
10 ³	rand1	16	1.62e+02	no	bt/maxit	0.000	0.04	27	5.59e-07	yes	–	1.577	0.08
10 ³	rand2	12	1.37e+02	no	bt/maxit	0.000	0.04	26	5.04e-08	yes	–	1.212	0.08
10 ³	rand3	10	2.01e+02	no	bt/maxit	0.000	0.02	25	6.04e-08	yes	–	0.904	0.07
10 ³	rand4	16	1.65e+02	no	bt/maxit	0.000	0.03	26	1.27e-07	yes	–	1.512	0.08
10 ³	rand5	12	1.61e+02	no	bt/maxit	0.000	0.03	25	9.98e-08	yes	–	1.458	0.08
10 ⁴	xbar	7	6.90e-08	yes	–	0.992	0.14	29	3.46e-06	no	maxit	0.000	0.76
10 ⁴	rand1	13	2.36e+03	no	bt/maxit	0.000	0.25	36	3.06e-08	yes	–	1.493	0.93
10 ⁴	rand2	17	3.45e+03	no	bt/maxit	0.000	0.30	35	2.57e-07	yes	–	1.519	0.90
10 ⁴	rand3	15	2.38e+03	no	bt/maxit	0.000	0.26	37	9.56e-08	yes	–	1.523	0.95
10 ⁴	rand4	17	2.15e+03	no	bt/maxit	0.000	0.29	39	4.46e-07	yes	–	1.506	0.99
10 ⁴	rand5	16	3.35e+03	no	bt/maxit	0.000	0.27	32	6.17e-07	yes	–	0.114	0.83
10 ⁵	xbar	7	6.90e-08	yes	–	0.992	1.16	29	2.81e-07	yes	–	1.501	8.16
10 ⁵	rand1	17	1.70e+05	no	bt/maxit	0.000	2.62	15	6.72e+04	no	bt/maxit	0.000	3.74
10 ⁵	rand2	22	5.43e+04	no	bt/maxit	0.000	3.43	38	1.72e-07	yes	–	1.504	10.59
10 ⁵	rand3	28	1.53e+04	no	bt/maxit	0.000	4.59	41	2.31e-07	yes	–	1.501	12.28
10 ⁵	rand4	9	3.21e+05	no	bt/maxit	0.000	1.56	48	1.03e-05	no	maxit	0.000	18.74
10 ⁵	rand5	16	1.60e+05	no	bt/maxit	0.000	2.64	16	1.11e+05	no	bt/maxit	0.000	4.60

Table 15: Trig16 – Tab k12_hi: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	5	1.69e-08	yes	–	1.617	0.00	5	6.96e-10	yes	–	2.024	0.00
2	rand1	4	2.14e-09	yes	–	1.669	0.00	5	3.59e-10	yes	–	1.808	0.00
2	rand2	5	6.42e-08	yes	–	2.061	0.00	5	1.48e-11	yes	–	1.556	0.00
2	rand3	5	1.24e-08	yes	–	1.530	0.00	5	1.17e-08	yes	–	1.499	0.00
2	rand4	4	7.23e-11	yes	–	0.959	0.00	5	1.58e-07	yes	–	NaN	0.00
2	rand5	3	3.40e-07	yes	–	2.823	0.00	6	3.63e-09	yes	–	2.141	0.00
10 ³	xbar	7	2.65e-09	yes	–	1.333	0.02	21	4.87e-08	yes	–	1.334	0.06
10 ³	rand1	14	5.11e-08	yes	–	1.647	0.02	27	6.54e-06	no	maxit	0.000	0.08
10 ³	rand2	16	4.62e-09	yes	–	0.621	0.03	24	4.94e-08	yes	–	0.882	0.07
10 ³	rand3	18	5.73e-09	yes	–	0.563	0.03	25	9.04e-08	yes	–	1.470	0.07
10 ³	rand4	17	8.59e-08	yes	–	2.126	0.04	26	5.19e-08	yes	–	0.787	0.07
10 ³	rand5	14	2.19e-07	yes	–	2.552	0.03	24	4.28e-08	yes	–	1.482	0.08
10 ⁴	xbar	7	2.65e-09	yes	–	1.333	0.12	25	1.35e-07	yes	–	1.527	0.64
10 ⁴	rand1	19	1.16e-07	yes	–	0.457	0.31	36	1.99e-07	yes	–	1.485	0.93
10 ⁴	rand2	18	5.48e-07	yes	–	0.209	0.30	32	4.03e-08	yes	–	1.510	0.83
10 ⁴	rand3	19	3.04e-07	yes	–	0.792	0.30	40	7.41e-06	no	maxit	0.000	1.05
10 ⁴	rand4	23	8.14e-07	yes	–	2.590	0.36	41	2.32e-05	no	maxit	0.000	1.06
10 ⁴	rand5	20	1.39e-07	yes	–	0.391	0.32	30	1.98e-08	yes	–	0.939	0.78
10 ⁵	xbar	7	2.65e-09	yes	–	1.333	1.23	35	1.58e-06	no	maxit	0.000	12.55
10 ⁵	rand1	50	9.07e-05	no	maxit	NaN	7.60	41	5.58e-04	no	maxit	0.000	13.77
10 ⁵	rand2	50	9.34e-05	no	maxit	NaN	7.41	38	3.17e-05	no	maxit	0.000	12.51
10 ⁵	rand3	50	8.23e-05	no	maxit	NaN	7.63	40	4.40e-08	yes	–	1.483	17.67
10 ⁵	rand4	50	6.77e-05	no	maxit	NaN	7.85	41	1.35e-05	no	maxit	0.000	21.51
10 ⁵	rand5	50	1.29e-05	no	maxit	NaN	7.60	44	2.96e-07	yes	–	1.501	21.80

Table 16: Broyden31 – Tab k4.h: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	5	$1.17e-07$	yes	–	2.001	0.00	7	$1.32e-07$	yes	–	1.991	0.02
2	rand1	6	$4.58e-08$	yes	–	1.609	0.00	8	$7.04e-09$	yes	–	1.987	0.00
2	rand2	6	$3.85e-10$	yes	–	1.074	0.00	7	$1.03e-12$	yes	–	1.993	0.00
2	rand3	6	$1.04e-07$	yes	–	1.848	0.00	8	$5.78e-08$	yes	–	1.991	0.00
2	rand4	7	$8.80e-08$	yes	–	2.290	0.00	7	$3.51e-07$	yes	–	1.981	0.00
2	rand5	6	$9.71e-07$	yes	–	2.132	0.00	6	$4.23e-13$	yes	–	1.997	0.00
10^3	xbar	5	$8.50e-08$	yes	–	1.463	0.03	5	$1.79e-08$	yes	–	1.760	0.00
10^3	rand1	9	$1.38e-09$	yes	–	1.140	0.05	9	$6.60e-12$	yes	–	1.992	0.00
10^3	rand2	9	$2.48e-07$	yes	–	2.120	0.03	9	$4.91e-07$	yes	–	1.978	0.00
10^3	rand3	9	$3.37e-07$	yes	–	2.217	0.03	9	$8.80e-07$	yes	–	1.982	0.01
10^3	rand4	9	$8.83e-09$	yes	–	1.276	0.04	9	$1.38e-10$	yes	–	1.970	0.00
10^3	rand5	2	$8.44e+02$	no	bt/maxit	NaN	0.01	2	$8.45e+02$	no	bt/maxit	NaN	0.00
10^4	xbar	5	$9.89e-08$	yes	–	1.242	0.17	5	$1.79e-08$	yes	–	1.559	0.02
10^4	rand1	10	$1.14e-08$	yes	–	1.328	0.40	10	$2.50e-10$	yes	–	1.999	0.04
10^4	rand2	11	$2.28e-07$	yes	–	2.033	0.45	11	$2.47e-07$	yes	–	1.990	0.05
10^4	rand3	11	$2.90e-10$	yes	–	1.058	0.39	11	$5.72e-13$	yes	–	1.999	0.05
10^4	rand4	10	$6.13e-10$	yes	–	1.079	0.35	10	$1.60e-12$	yes	–	1.999	0.05
10^4	rand5	10	$8.83e-09$	yes	–	1.295	0.34	10	$1.26e-10$	yes	–	1.999	0.04
10^5	xbar	5	$1.88e-07$	yes	–	1.086	1.90	5	$1.79e-08$	yes	–	1.555	0.27
10^5	rand1	12	$2.74e-08$	yes	–	1.425	3.87	12	$6.76e-10$	yes	–	1.983	0.79
10^5	rand2	12	$3.32e-10$	yes	–	1.054	4.11	12	$2.18e-13$	yes	–	1.999	0.65
10^5	rand3	12	$9.21e-09$	yes	–	1.300	4.09	12	$1.48e-10$	yes	–	2.001	0.70
10^5	rand4	12	$1.12e-08$	yes	–	1.321	4.03	12	$1.67e-10$	yes	–	1.999	0.66
10^5	rand5	10	$1.07e-08$	yes	–	1.320	3.45	10	$2.15e-10$	yes	–	2.000	0.77

Table 17: Broyden31 – Tab k4.hi: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	5	$1.17e-07$	yes	–	2.001	0.00	7	$1.32e-07$	yes	–	1.991	0.02
2	rand1	6	$4.58e-08$	yes	–	1.609	0.00	8	$7.04e-09$	yes	–	1.987	0.00
2	rand2	6	$3.85e-10$	yes	–	1.074	0.00	7	$1.03e-12$	yes	–	1.993	0.00
2	rand3	6	$1.04e-07$	yes	–	1.848	0.00	8	$5.78e-08$	yes	–	1.991	0.00
2	rand4	7	$8.80e-08$	yes	–	2.290	0.00	7	$3.51e-07$	yes	–	1.981	0.00
2	rand5	6	$9.71e-07$	yes	–	2.132	0.00	6	$4.23e-13$	yes	–	1.997	0.00
10^3	xbar	5	$8.50e-08$	yes	–	1.463	0.02	5	$1.79e-08$	yes	–	1.760	0.00
10^3	rand1	9	$1.38e-09$	yes	–	1.140	0.03	9	$6.60e-12$	yes	–	1.992	0.00
10^3	rand2	9	$2.48e-07$	yes	–	2.119	0.03	9	$4.91e-07$	yes	–	1.978	0.00
10^3	rand3	9	$3.37e-07$	yes	–	2.217	0.03	9	$8.80e-07$	yes	–	1.982	0.01
10^3	rand4	9	$8.81e-09$	yes	–	1.276	0.03	9	$1.38e-10$	yes	–	1.970	0.00
10^3	rand5	2	$8.44e+02$	no	bt/maxit	NaN	0.01	2	$8.45e+02$	no	bt/maxit	NaN	0.00
10^4	xbar	5	$9.89e-08$	yes	–	1.242	0.18	5	$1.79e-08$	yes	–	1.559	0.03
10^4	rand1	10	$1.14e-08$	yes	–	1.328	0.35	10	$2.50e-10$	yes	–	2.000	0.04
10^4	rand2	11	$2.28e-07$	yes	–	2.032	0.35	11	$2.47e-07$	yes	–	1.990	0.05
10^4	rand3	11	$2.88e-10$	yes	–	1.058	0.37	11	$5.72e-13$	yes	–	1.999	0.05
10^4	rand4	10	$6.11e-10$	yes	–	1.079	0.33	10	$1.60e-12$	yes	–	1.999	0.05
10^4	rand5	10	$8.80e-09$	yes	–	1.295	0.34	10	$1.26e-10$	yes	–	1.999	0.04
10^5	xbar	5	$1.88e-07$	yes	–	1.086	1.70	5	$1.79e-08$	yes	–	1.555	0.27
10^5	rand1	12	$2.72e-08$	yes	–	1.424	3.87	12	$6.76e-10$	yes	–	1.983	0.79
10^5	rand2	12	$3.28e-10$	yes	–	1.053	3.91	12	$2.18e-13$	yes	–	1.999	0.65
10^5	rand3	12	$9.18e-09$	yes	–	1.299	3.94	12	$1.48e-10$	yes	–	2.001	0.70
10^5	rand4	12	$1.12e-08$	yes	–	1.321	4.21	12	$1.67e-10$	yes	–	1.999	0.66
10^5	rand5	10	$1.07e-08$	yes	–	1.320	3.55	10	$2.15e-10$	yes	–	2.000	0.77

Table 18: Broyden31 – Tab k8.h: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	5	$1.32e-07$	yes	–	1.990	0.00	7	$1.32e-07$	yes	–	1.991	0.00
2	rand1	6	$7.03e-09$	yes	–	1.987	0.00	8	$7.04e-09$	yes	–	1.987	0.00
2	rand2	6	$9.63e-13$	yes	–	2.003	0.00	7	$1.03e-12$	yes	–	1.993	0.00
2	rand3	6	$5.77e-08$	yes	–	1.991	0.00	8	$5.78e-08$	yes	–	1.991	0.00
2	rand4	7	$3.51e-07$	yes	–	1.981	0.00	7	$3.51e-07$	yes	–	1.981	0.00
2	rand5	7	$3.80e-13$	yes	–	2.011	0.00	6	$4.23e-13$	yes	–	1.997	0.00
10^3	xbar	5	$1.79e-08$	yes	–	1.760	0.02	5	$1.79e-08$	yes	–	1.760	0.00
10^3	rand1	9	$6.46e-12$	yes	–	1.996	0.03	9	$6.60e-12$	yes	–	1.992	0.00
10^3	rand2	9	$4.91e-07$	yes	–	1.978	0.03	9	$4.91e-07$	yes	–	1.978	0.00
10^3	rand3	9	$8.80e-07$	yes	–	1.982	0.04	9	$8.80e-07$	yes	–	1.982	0.01
10^3	rand4	9	$1.38e-10$	yes	–	1.971	0.03	9	$1.38e-10$	yes	–	1.970	0.00
10^3	rand5	2	$8.45e+02$	no	bt/maxit	NaN	0.01	2	$8.45e+02$	no	bt/maxit	NaN	0.00
10^4	xbar	5	$1.79e-08$	yes	–	1.559	0.17	5	$1.79e-08$	yes	–	1.559	0.02
10^4	rand1	10	$2.49e-10$	yes	–	2.001	0.33	10	$2.50e-10$	yes	–	2.000	0.04
10^4	rand2	11	$2.47e-07$	yes	–	1.990	0.37	11	$2.47e-07$	yes	–	1.990	0.05
10^4	rand3	11	$5.08e-13$	yes	–	2.014	0.36	11	$5.72e-13$	yes	–	1.999	0.05
10^4	rand4	10	$1.49e-12$	yes	–	2.007	0.33	10	$1.60e-12$	yes	–	1.999	0.05
10^4	rand5	10	$1.25e-10$	yes	–	2.000	0.34	10	$1.26e-10$	yes	–	1.999	0.04
10^5	xbar	5	$1.79e-08$	yes	–	1.555	1.69	5	$1.79e-08$	yes	–	1.555	0.27
10^5	rand1	12	$6.76e-10$	yes	–	1.983	3.82	12	$6.76e-10$	yes	–	1.983	0.79
10^5	rand2	12	$1.88e-13$	yes	–	2.018	3.85	12	$2.18e-13$	yes	–	1.999	0.65
10^5	rand3	12	$1.47e-10$	yes	–	2.002	4.11	12	$1.48e-10$	yes	–	2.001	0.70
10^5	rand4	12	$1.65e-10$	yes	–	2.001	4.47	12	$1.67e-10$	yes	–	1.999	0.66
10^5	rand5	10	$2.14e-10$	yes	–	2.001	3.81	10	$2.15e-10$	yes	–	2.000	0.77

Table 19: Broyden31 – Tab k8.hi: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	5	$1.32e-07$	yes	–	1.990	0.00	7	$1.32e-07$	yes	–	1.991	0.00
2	rand1	6	$7.03e-09$	yes	–	1.987	0.00	8	$7.04e-09$	yes	–	1.987	0.00
2	rand2	6	$9.52e-13$	yes	–	2.004	0.00	7	$1.03e-12$	yes	–	1.993	0.00
2	rand3	6	$5.77e-08$	yes	–	1.991	0.00	8	$5.78e-08$	yes	–	1.991	0.00
2	rand4	7	$3.51e-07$	yes	–	1.981	0.00	7	$3.51e-07$	yes	–	1.981	0.00
2	rand5	7	$3.61e-13$	yes	–	2.018	0.00	6	$4.23e-13$	yes	–	1.997	0.00
10^3	xbar	5	$1.79e-08$	yes	–	1.760	0.02	5	$1.79e-08$	yes	–	1.760	0.00
10^3	rand1	9	$6.42e-12$	yes	–	1.996	0.03	9	$6.60e-12$	yes	–	1.992	0.00
10^3	rand2	9	$4.91e-07$	yes	–	1.978	0.03	9	$4.91e-07$	yes	–	1.978	0.00
10^3	rand3	9	$8.80e-07$	yes	–	1.982	0.03	9	$8.80e-07$	yes	–	1.982	0.01
10^3	rand4	9	$1.38e-10$	yes	–	1.971	0.03	9	$1.38e-10$	yes	–	1.970	0.00
10^3	rand5	2	$8.45e+02$	no	bt/maxit	NaN	0.01	2	$8.45e+02$	no	bt/maxit	NaN	0.00
10^4	xbar	5	$1.79e-08$	yes	–	1.559	0.17	5	$1.79e-08$	yes	–	1.559	0.02
10^4	rand1	10	$2.50e-10$	yes	–	2.000	0.34	10	$2.50e-10$	yes	–	2.000	0.04
10^4	rand2	11	$2.47e-07$	yes	–	1.990	0.40	11	$2.47e-07$	yes	–	1.990	0.05
10^4	rand3	11	$5.14e-13$	yes	–	2.013	0.36	11	$5.72e-13$	yes	–	1.999	0.05
10^4	rand4	10	$1.53e-12$	yes	–	2.004	0.34	10	$1.60e-12$	yes	–	1.999	0.05
10^4	rand5	10	$1.25e-10$	yes	–	1.999	0.34	10	$1.26e-10$	yes	–	1.999	0.04
10^5	xbar	5	$1.79e-08$	yes	–	1.555	2.04	5	$1.79e-08$	yes	–	1.555	0.27
10^5	rand1	12	$6.76e-10$	yes	–	1.983	4.16	12	$6.76e-10$	yes	–	1.983	0.79
10^5	rand2	12	$1.83e-13$	yes	–	2.021	3.93	12	$2.18e-13$	yes	–	1.999	0.65
10^5	rand3	12	$1.48e-10$	yes	–	2.002	4.32	12	$1.48e-10$	yes	–	2.001	0.70
10^5	rand4	12	$1.66e-10$	yes	–	2.000	4.07	12	$1.67e-10$	yes	–	1.999	0.66
10^5	rand5	10	$2.15e-10$	yes	–	2.001	3.67	10	$2.15e-10$	yes	–	2.000	0.77

Table 20: Broyden31 – Tab k12.h: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	5	$2.25e-07$	yes	–	1.869	0.00	7	$8.71e-10$	yes	–	1.591	0.00
2	rand1	6	$1.51e-08$	yes	–	1.848	0.00	8	$5.76e-09$	yes	–	1.603	0.00
2	rand2	6	$1.46e-10$	yes	–	1.292	0.00	7	$2.16e-08$	yes	–	1.856	0.00
2	rand3	6	$3.08e-08$	yes	–	2.121	0.00	8	$1.89e-09$	yes	–	1.764	0.00
2	rand4	7	$3.08e-07$	yes	–	2.013	0.00	7	$8.42e-08$	yes	–	1.977	0.00
2	rand5	7	$2.32e-11$	yes	–	1.443	0.00	6	$4.60e-08$	yes	–	1.984	0.00
10^3	xbar	5	$1.69e-08$	yes	–	1.771	0.02	13	$1.91e-07$	yes	–	1.185	0.00
10^3	rand1	9	$8.44e-10$	yes	–	1.307	0.04	15	$1.56e-07$	yes	–	0.879	0.01
10^3	rand2	9	$4.72e-07$	yes	–	1.993	0.03	15	$6.72e-07$	yes	–	0.820	0.00
10^3	rand3	9	$8.70e-07$	yes	–	2.008	0.03	15	$2.27e-07$	yes	–	1.016	0.00
10^3	rand4	9	$1.22e-08$	yes	–	1.282	0.03	15	$1.44e-07$	yes	–	1.154	0.00
10^3	rand5	2	$8.44e+02$	no	bt/maxit	NaN	0.01	15	$1.76e-07$	yes	–	0.952	0.00
10^4	xbar	5	$3.06e-08$	yes	–	1.495	0.17	13	$6.24e-07$	yes	–	0.793	0.03
10^4	rand1	10	$9.68e-09$	yes	–	1.397	0.35	16	$6.68e-07$	yes	–	0.936	0.03
10^4	rand2	11	$3.82e-07$	yes	–	1.861	0.37	16	$3.95e-07$	yes	–	0.824	0.05
10^4	rand3	11	$2.22e-10$	yes	–	1.186	0.38	16	$1.32e-07$	yes	–	1.028	0.04
10^4	rand4	10	$6.81e-10$	yes	–	1.188	0.36	17	$5.02e-07$	yes	–	0.891	0.04
10^4	rand5	10	$6.63e-09$	yes	–	1.390	0.38	16	$6.56e-07$	yes	–	0.906	0.04
10^5	xbar	5	$8.64e-08$	yes	–	1.372	1.80	14	$3.71e-07$	yes	–	0.861	0.34
10^5	rand1	12	$1.02e-08$	yes	–	1.496	4.14	17	$1.90e-07$	yes	–	1.069	0.41
10^5	rand2	12	$2.85e-10$	yes	–	0.998	3.92	17	$9.05e-08$	yes	–	0.787	0.42
10^5	rand3	12	$5.09e-09$	yes	–	1.435	3.96	17	$2.43e-07$	yes	–	1.002	0.43
10^5	rand4	12	$1.39e-08$	yes	–	1.317	4.15	17	$9.73e-08$	yes	–	0.815	0.43
10^5	rand5	10	$4.95e-09$	yes	–	1.456	3.60	17	$1.04e-07$	yes	–	0.816	0.42

Table 21: Broyden31 – Tab k12.hi: Modified Newton vs Truncated Newton with exact gradient and approximated hessian (CASE 1).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	5	$2.25e-07$	yes	–	1.869	0.00	7	$8.71e-10$	yes	–	1.591	0.00
2	rand1	6	$3.38e-09$	yes	–	2.121	0.00	8	$1.13e-09$	yes	–	1.879	0.00
2	rand2	6	$1.81e-10$	yes	–	1.279	0.00	7	$1.02e-08$	yes	–	2.010	0.00
2	rand3	6	$1.84e-08$	yes	–	2.229	0.00	8	$1.18e-08$	yes	–	1.437	0.00
2	rand4	7	$3.70e-07$	yes	–	1.970	0.00	7	$1.07e-07$	yes	–	1.923	0.00
2	rand5	7	$1.48e-10$	yes	–	1.206	0.00	6	$5.36e-08$	yes	–	1.950	0.00
10^3	xbar	5	$1.69e-08$	yes	–	1.771	0.02	13	$1.91e-07$	yes	–	1.185	0.00
10^3	rand1	9	$6.28e-10$	yes	–	1.363	0.03	15	$1.56e-07$	yes	–	0.879	0.01
10^3	rand2	9	$7.34e-07$	yes	–	1.898	0.03	15	$6.73e-07$	yes	–	0.820	0.00
10^3	rand3	9	$7.69e-07$	yes	–	2.024	0.03	15	$2.27e-07$	yes	–	1.016	0.00
10^3	rand4	9	$4.88e-09$	yes	–	1.410	0.03	15	$1.44e-07$	yes	–	1.154	0.00
10^3	rand5	2	$8.44e+02$	no	bt/maxit	NaN	0.01	15	$1.76e-07$	yes	–	0.951	0.00
10^4	xbar	5	$3.06e-08$	yes	–	1.495	0.18	13	$6.24e-07$	yes	–	0.793	0.03
10^4	rand1	10	$9.44e-09$	yes	–	1.389	0.34	16	$6.68e-07$	yes	–	0.936	0.03
10^4	rand2	11	$3.26e-07$	yes	–	1.896	0.35	16	$3.95e-07$	yes	–	0.824	0.05
10^4	rand3	11	$1.89e-10$	yes	–	1.208	0.35	16	$1.32e-07$	yes	–	1.028	0.04
10^4	rand4	10	$6.54e-10$	yes	–	1.134	0.33	17	$5.02e-07$	yes	–	0.891	0.04
10^4	rand5	10	$8.32e-09$	yes	–	1.340	0.32	16	$6.56e-07$	yes	–	0.906	0.04
10^5	xbar	5	$8.64e-08$	yes	–	1.372	1.73	14	$3.72e-07$	yes	–	0.862	0.34
10^5	rand1	12	$1.39e-08$	yes	–	1.489	4.18	17	$1.89e-07$	yes	–	1.070	0.41
10^5	rand2	12	$3.05e-10$	yes	–	1.079	4.20	17	$9.06e-08$	yes	–	0.787	0.42
10^5	rand3	12	$4.78e-09$	yes	–	1.410	4.27	17	$2.43e-07$	yes	–	1.002	0.43
10^5	rand4	12	$6.25e-09$	yes	–	1.439	4.39	17	$9.73e-08$	yes	–	0.815	0.43
10^5	rand5	10	$4.48e-09$	yes	–	1.510	3.55	17	$1.04e-07$	yes	–	0.816	0.42

Table 22: Trig16 – Tab k4_h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	3	1.26e+00	no	bt/maxit	0.000	0.05	3	1.04e+00	no	maxit	0.000	0.00
2	rand1	2	2.25e+00	no	bt/maxit	0.000	0.01	3	2.89e+00	no	maxit	0.000	0.00
2	rand2	2	1.78e+00	no	bt/maxit	0.000	0.00	3	6.26e-01	no	maxit	0.000	0.00
2	rand3	1	2.51e+00	no	bt/maxit	0.000	0.01	2	2.51e+00	no	maxit	NaN	0.00
2	rand4	1	4.63e-01	no	bt/maxit	0.000	0.00	2	4.63e-01	no	maxit	NaN	0.00
2	rand5	2	3.97e-01	no	bt/maxit	0.000	0.00	4	4.72e-01	no	maxit	0.000	0.00
10 ³	xbar	7	2.12e+03	no	bt/maxit	0.000	0.01	8	2.53e+03	no	maxit	0.000	0.00
10 ³	rand1	15	2.00e+03	no	bt/maxit	0.000	0.01	24	9.12e+03	no	maxit	0.000	0.01
10 ³	rand2	12	1.73e+03	no	bt/maxit	0.000	0.01	18	1.13e+04	no	maxit	0.000	0.01
10 ³	rand3	14	2.38e+03	no	bt/maxit	0.000	0.01	51	8.77e+03	no	maxit	0.681	0.02
10 ³	rand4	8	1.71e+03	no	bt/maxit	0.000	0.00	50	9.01e+03	no	maxit	0.000	0.02
10 ³	rand5	9	2.11e+03	no	bt/maxit	0.000	0.00	38	9.65e+03	no	maxit	0.000	0.02
10 ⁴	xbar	3	1.15e+04	no	bt/maxit	0.000	0.01	30	6.79e+03	no	maxit	0.000	0.04
10 ⁴	rand1	8	7.07e+04	no	bt/maxit	0.000	0.03	51	3.06e+05	no	maxit	NaN	0.11
10 ⁴	rand2	14	5.38e+04	no	bt/maxit	0.000	0.08	51	2.95e+05	no	maxit	0.691	0.11
10 ⁴	rand3	10	4.25e+04	no	bt/maxit	0.000	0.03	51	2.83e+05	no	maxit	NaN	0.11
10 ⁴	rand4	10	5.10e+04	no	bt/maxit	0.000	0.03	51	2.73e+05	no	maxit	0.130	0.11
10 ⁴	rand5	11	5.00e+04	no	bt/maxit	0.000	0.03	51	2.96e+05	no	maxit	1.516	0.11
10 ⁵	xbar	3	1.81e+06	no	bt/maxit	0.000	0.10	8	3.44e+05	no	maxit	0.000	0.15
10 ⁵	rand1	15	1.08e+06	no	bt/maxit	0.000	0.48	51	8.81e+06	no	maxit	NaN	0.93
10 ⁵	rand2	13	1.59e+06	no	bt/maxit	0.000	0.37	51	9.03e+06	no	maxit	0.611	1.02
10 ⁵	rand3	10	3.28e+06	no	bt/maxit	0.000	0.36	51	8.49e+06	no	maxit	0.933	0.97
10 ⁵	rand4	8	3.59e+06	no	bt/maxit	0.000	0.31	51	9.00e+06	no	maxit	NaN	0.93
10 ⁵	rand5	9	2.06e+06	no	bt/maxit	0.000	0.27	51	8.96e+06	no	maxit	1.149	1.01

Table 23: Trig16 – k4_hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	3	2.74e+02	no	bt/maxit	0.000	0.03	3	1.04e+00	no	maxit	0.000	0.00
2	rand1	2	2.77e+00	no	bt/maxit	0.000	0.00	3	1.77e+00	no	maxit	0.000	0.00
2	rand2	4	1.11e+01	no	bt/maxit	0.000	0.00	3	4.40e-01	no	maxit	0.000	0.00
2	rand3	1	2.51e+00	no	bt/maxit	0.000	0.00	2	2.51e+00	no	maxit	NaN	0.00
2	rand4	1	5.64e-01	no	bt/maxit	0.000	0.00	3	6.70e-01	no	maxit	0.000	0.00
2	rand5	6	2.24e+00	no	bt/maxit	0.000	0.00	3	1.19e+00	no	maxit	0.000	0.00
10 ³	xbar	2	8.94e+06	no	bt/maxit	0.000	0.00	10	1.09e+04	no	maxit	0.000	0.01
10 ³	rand1	2	4.07e+06	no	bt/maxit	0.000	0.00	7	7.14e+03	no	maxit	0.000	0.00
10 ³	rand2	2	6.77e+06	no	bt/maxit	0.000	0.00	6	2.79e+04	no	maxit	0.000	0.01
10 ³	rand3	2	3.25e+05	no	bt/maxit	0.000	0.00	51	1.68e+04	no	maxit	0.214	0.02
10 ³	rand4	2	6.30e+06	no	bt/maxit	0.000	0.00	12	1.30e+04	no	maxit	0.000	0.01
10 ³	rand5	2	3.18e+06	no	bt/maxit	0.000	0.00	13	2.54e+04	no	maxit	0.000	0.01
10 ⁴	xbar	3	1.88e+07	no	bt/maxit	0.000	0.01	11	7.98e+04	no	maxit	0.000	0.03
10 ⁴	rand1	2	6.24e+07	no	bt/maxit	0.000	0.01	12	5.41e+05	no	maxit	0.000	0.04
10 ⁴	rand2	2	8.19e+06	no	bt/maxit	0.000	0.01	18	3.55e+05	no	maxit	0.000	0.05
10 ⁴	rand3	2	9.46e+05	no	bt/maxit	0.000	0.01	11	4.46e+05	no	maxit	0.000	0.05
10 ⁴	rand4	2	5.21e+07	no	bt/maxit	0.000	0.01	11	5.95e+05	no	maxit	0.000	0.03
10 ⁴	rand5	2	1.01e+08	no	bt/maxit	0.000	0.01	11	5.65e+05	no	maxit	0.000	0.03
10 ⁵	xbar	2	2.48e+08	no	bt/maxit	0.000	0.06	16	5.60e+05	no	maxit	0.000	0.27
10 ⁵	rand1	2	1.85e+08	no	bt/maxit	0.000	0.08	23	1.20e+07	no	maxit	0.000	0.70
10 ⁵	rand2	2	7.22e+07	no	bt/maxit	0.000	0.10	10	1.28e+07	no	maxit	0.000	0.22
10 ⁵	rand3	2	8.84e+08	no	bt/maxit	0.000	0.12	5	1.19e+07	no	maxit	0.000	0.19
10 ⁵	rand4	2	1.96e+08	no	bt/maxit	0.000	0.11	6	7.56e+06	no	maxit	0.000	0.17
10 ⁵	rand5	2	5.87e+08	no	bt/maxit	0.000	0.13	5	1.29e+07	no	maxit	0.000	0.15

Table 24: Trig16 – k8_h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	3	2.48e+00	no	bt/maxit	0.000	0.00	3	3.27e+00	no	maxit	0.000	0.00
2	rand1	2	2.85e+00	no	bt/maxit	0.000	0.00	3	2.59e+00	no	maxit	0.000	0.00
2	rand2	4	3.25e+00	no	bt/maxit	0.000	0.00	4	2.49e+00	no	maxit	0.000	0.00
2	rand3	3	2.73e+00	no	bt/maxit	0.000	0.00	2	2.51e+00	no	maxit	NaN	0.00
2	rand4	1	4.63e-01	no	bt/maxit	0.000	0.00	2	4.63e-01	no	maxit	NaN	0.00
2	rand5	6	2.52e+00	no	bt/maxit	0.000	0.00	3	5.04e-02	no	maxit	0.000	0.00
10 ³	xbar	6	6.43e+03	no	bt/maxit	0.000	0.00	18	2.88e+03	no	maxit	0.000	0.01
10 ³	rand1	20	4.45e+03	no	bt/maxit	0.000	0.01	21	7.97e+03	no	maxit	0.000	0.01
10 ³	rand2	15	6.72e+03	no	bt/maxit	0.000	0.01	31	8.71e+03	no	maxit	0.000	0.02
10 ³	rand3	9	1.16e+04	no	bt/maxit	0.000	0.01	17	7.35e+03	no	maxit	0.000	0.01
10 ³	rand4	31	2.71e+03	no	bt/maxit	0.000	0.02	51	7.81e+03	no	maxit	NaN	0.03
10 ³	rand5	19	5.11e+03	no	bt/maxit	0.000	0.01	51	7.82e+03	no	maxit	0.237	0.03
10 ⁴	xbar	1	4.86e+05	no	bt/maxit	0.000	0.01	2	4.86e+05	no	maxit	NaN	0.12
10 ⁴	rand1	23	9.96e+04	no	bt/maxit	0.000	0.06	51	2.40e+05	no	maxit	NaN	0.16
10 ⁴	rand2	20	2.32e+05	no	bt/maxit	0.000	0.07	44	3.00e+05	no	maxit	0.000	0.15
10 ⁴	rand3	25	9.58e+04	no	bt/maxit	0.000	0.08	51	2.85e+05	no	maxit	NaN	0.17
10 ⁴	rand4	17	2.37e+05	no	bt/maxit	0.000	0.05	30	3.06e+05	no	maxit	0.000	0.08
10 ⁴	rand5	19	2.39e+05	no	bt/maxit	0.000	0.06	39	2.88e+05	no	maxit	0.000	0.11
10 ⁵	xbar	6	7.27e+06	no	bt/maxit	0.000	0.27	51	4.59e+06	no	maxit	NaN	0.89
10 ⁵	rand1	8	1.49e+07	no	bt/maxit	0.000	0.29	51	8.18e+06	no	maxit	0.746	1.00
10 ⁵	rand2	21	9.55e+06	no	bt/maxit	0.000	0.87	51	7.45e+06	no	maxit	NaN	0.98
10 ⁵	rand3	21	6.51e+06	no	bt/maxit	0.000	0.53	51	8.10e+06	no	maxit	2.425	0.96
10 ⁵	rand4	5	1.63e+07	no	bt/maxit	0.000	0.21	51	7.94e+06	no	maxit	0.388	0.98
10 ⁵	rand5	24	4.32e+06	no	bt/maxit	0.000	0.64	51	7.18e+06	no	maxit	1.103	0.88

Table 25: Trig16 – k8_hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	3	2.74e+02	no	bt/maxit	0.000	0.00	3	8.86e+00	no	maxit	0.000	0.00
2	rand1	2	1.65e+00	no	bt/maxit	0.000	0.00	3	1.77e+00	no	maxit	0.000	0.00
2	rand2	5	2.52e+00	no	bt/maxit	0.000	0.00	3	4.42e+00	no	maxit	0.000	0.00
2	rand3	4	5.42e+00	no	bt/maxit	0.000	0.00	2	2.51e+00	no	maxit	NaN	0.00
2	rand4	1	5.64e-01	no	bt/maxit	0.000	0.00	3	6.70e-01	no	maxit	0.000	0.00
2	rand5	3	9.46e+01	no	bt/maxit	0.000	0.00	3	1.19e+00	no	maxit	0.000	0.00
10 ³	xbar	2	2.75e+08	no	bt/maxit	0.000	0.00	9	4.23e+03	no	maxit	0.000	0.00
10 ³	rand1	2	4.36e+06	no	bt/maxit	0.000	0.00	12	8.95e+03	no	maxit	0.000	0.01
10 ³	rand2	2	2.73e+07	no	bt/maxit	0.000	0.00	23	7.26e+03	no	maxit	0.000	0.01
10 ³	rand3	2	7.78e+06	no	bt/maxit	0.000	0.00	12	7.19e+03	no	maxit	0.000	0.01
10 ³	rand4	2	1.18e+07	no	bt/maxit	0.000	0.00	17	4.90e+02	no	maxit	0.000	0.01
10 ³	rand5	2	7.20e+07	no	bt/maxit	0.000	0.00	12	2.49e+03	no	maxit	0.000	0.00
10 ⁴	xbar	3	1.88e+07	no	bt/maxit	0.000	0.01	2	4.86e+05	no	maxit	NaN	0.11
10 ⁴	rand1	2	6.24e+07	no	bt/maxit	0.000	0.01	10	2.41e+05	no	maxit	0.000	0.04
10 ⁴	rand2	2	8.19e+06	no	bt/maxit	0.000	0.01	11	2.01e+05	no	maxit	0.000	0.03
10 ⁴	rand3	2	9.46e+05	no	bt/maxit	0.000	0.01	13	1.29e+05	no	maxit	0.000	0.03
10 ⁴	rand4	2	5.21e+07	no	bt/maxit	0.000	0.01	20	2.10e+05	no	maxit	0.000	0.03
10 ⁴	rand5	2	1.01e+08	no	bt/maxit	0.000	0.01	14	1.22e+05	no	maxit	0.000	0.03
10 ⁵	xbar	16	5.60e+05	no	bt/maxit	0.000	0.27	15	1.62e+07	no	maxit	0.000	0.26
10 ⁵	rand1	23	1.20e+07	no	bt/maxit	0.000	0.70	10	2.69e+06	no	maxit	0.000	0.21
10 ⁵	rand2	10	1.28e+07	no	bt/maxit	0.000	0.22	8	3.64e+06	no	maxit	0.000	0.17
10 ⁵	rand3	5	1.19e+07	no	bt/maxit	0.000	0.19	24	2.70e+06	no	maxit	0.000	0.36
10 ⁵	rand4	6	7.56e+06	no	bt/maxit	0.000	0.17	7	4.90e+06	no	maxit	0.000	0.16
10 ⁵	rand5	5	1.29e+07	no	bt/maxit	0.000	0.15	9	3.30e+06	no	maxit	0.000	0.18

Table 26: Trig16 – Tab k12.h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	3	2.53e+00	no	bt/maxit	0.000	0.00	3	1.04e+00	no	maxit	0.000	0.00
2	rand1	5	1.83e+00	no	bt/maxit	0.000	0.00	3	1.88e+00	no	maxit	0.000	0.00
2	rand2	5	3.48e+00	no	bt/maxit	0.000	0.00	3	6.25e-01	no	maxit	0.000	0.00
2	rand3	1	2.51e+00	no	bt/maxit	0.000	0.00	2	2.51e+00	no	maxit	NaN	0.00
2	rand4	4	2.55e+00	no	bt/maxit	0.000	0.00	2	4.63e-01	no	maxit	NaN	0.00
2	rand5	7	3.12e+00	no	bt/maxit	0.000	0.00	5	5.03e-02	no	maxit	0.000	0.00
10 ³	xbar	1	1.54e+04	no	bt/maxit	0.000	0.00	21	2.45e+03	no	maxit	0.000	0.01
10 ³	rand1	28	1.81e+04	no	bt/maxit	0.000	0.02	51	1.03e+04	no	maxit	0.167	0.03
10 ³	rand2	13	1.82e+04	no	bt/maxit	0.000	0.01	51	1.26e+04	no	maxit	0.000	0.02
10 ³	rand3	16	1.84e+04	no	bt/maxit	0.000	0.01	51	1.04e+04	no	maxit	NaN	0.03
10 ³	rand4	35	1.79e+04	no	bt/maxit	0.000	0.02	51	1.25e+04	no	maxit	NaN	0.02
10 ³	rand5	6	1.85e+04	no	bt/maxit	0.000	0.00	51	1.07e+04	no	maxit	NaN	0.03
10 ⁴	xbar	1	4.86e+05	no	bt/maxit	0.000	0.01	25	1.54e+05	no	maxit	0.000	0.11
10 ⁴	rand1	7	5.76e+05	no	bt/maxit	0.000	0.03	38	3.48e+05	no	maxit	0.000	0.14
10 ⁴	rand2	8	5.72e+05	no	bt/maxit	0.000	0.03	40	3.76e+05	no	maxit	0.000	0.16
10 ⁴	rand3	13	5.73e+05	no	bt/maxit	0.000	0.08	35	3.61e+05	no	maxit	0.000	0.16
10 ⁴	rand4	18	5.75e+05	no	bt/maxit	0.000	0.08	28	3.69e+05	no	maxit	0.000	0.12
10 ⁴	rand5	11	5.79e+05	no	bt/maxit	0.000	0.06	19	3.86e+05	no	maxit	0.000	0.11
10 ⁵	xbar	2	1.54e+07	no	bt/maxit	0.000	0.08	2	1.54e+07	no	maxit	NaN	0.12
10 ⁵	rand1	7	1.82e+07	no	bt/maxit	0.000	0.33	7	1.82e+07	no	maxit	0.000	0.20
10 ⁵	rand2	1	1.82e+07	no	bt/maxit	0.000	0.07	4	1.82e+07	no	maxit	0.000	0.15
10 ⁵	rand3	14	1.82e+07	no	bt/maxit	0.000	0.60	10	1.82e+07	no	maxit	0.000	0.24
10 ⁵	rand4	14	1.82e+07	no	bt/maxit	0.000	0.75	4	1.82e+07	no	maxit	0.000	0.18
10 ⁵	rand5	36	1.82e+07	no	bt/maxit	0.000	1.34	3	1.82e+07	no	maxit	0.000	0.15

Table 27: Trig16 – Tab k12.hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	3	7.47e+00	no	bt/maxit	0.000	0.00	3	1.04e+00	no	maxit	0.000	0.00
2	rand1	2	2.33e+01	no	bt/maxit	0.000	0.00	3	2.02e+00	no	maxit	0.000	0.00
2	rand2	2	3.20e+01	no	bt/maxit	0.000	0.00	6	4.40e-01	no	maxit	0.000	0.00
2	rand3	1	2.51e+00	no	bt/maxit	0.000	0.00	2	2.51e+00	no	maxit	NaN	0.00
2	rand4	1	5.64e-01	no	bt/maxit	0.000	0.00	3	6.70e-01	no	maxit	0.000	0.00
2	rand5	3	1.07e+02	no	bt/maxit	0.000	0.00	3	1.19e+00	no	maxit	0.000	0.00
10 ³	xbar	1	1.54e+04	no	bt/maxit	0.000	0.00	25	5.05e+03	no	maxit	0.000	0.02
10 ³	rand1	2	8.29e+05	no	bt/maxit	0.000	0.00	51	1.50e+04	no	maxit	NaN	0.04
10 ³	rand2	1	1.84e+04	no	bt/maxit	0.000	0.00	35	1.75e+04	no	maxit	0.000	0.03
10 ³	rand3	2	8.76e+06	no	bt/maxit	0.000	0.00	51	2.38e+04	no	maxit	NaN	0.03
10 ³	rand4	4	1.09e+06	no	bt/maxit	0.000	0.00	30	2.74e+04	no	maxit	0.000	0.02
10 ³	rand5	1	1.82e+04	no	bt/maxit	0.000	0.00	51	1.50e+04	no	maxit	NaN	0.03
10 ⁴	xbar	27	6.34e+05	no	bt/maxit	0.000	0.12	27	6.34e+05	no	maxit	0.000	0.12
10 ⁴	rand1	51	4.94e+05	no	bt/maxit	0.000	0.23	51	4.94e+05	no	maxit	0.149	0.23
10 ⁴	rand2	18	9.30e+05	no	bt/maxit	0.000	0.09	18	9.30e+05	no	maxit	0.000	0.09
10 ⁴	rand3	51	8.01e+05	no	bt/maxit	0.000	0.25	51	8.01e+05	no	maxit	NaN	0.25
10 ⁴	rand4	31	6.66e+05	no	bt/maxit	0.000	0.19	31	6.66e+05	no	maxit	0.000	0.19
10 ⁴	rand5	51	4.82e+05	no	bt/maxit	0.000	0.25	51	4.82e+05	no	maxit	0.449	0.25
10 ⁵	xbar	2	1.54e+07	no	bt/maxit	0.000	0.13	2	1.54e+07	no	maxit	NaN	0.13
10 ⁵	rand1	1	1.82e+07	no	bt/maxit	0.000	0.04	2	1.82e+07	no	maxit	NaN	0.13
10 ⁵	rand2	1	1.82e+07	no	bt/maxit	0.000	0.04	2	1.82e+07	no	maxit	NaN	0.12
10 ⁵	rand3	1	1.82e+07	no	bt/maxit	0.000	0.06	6	1.82e+07	no	maxit	0.000	0.20
10 ⁵	rand4	1	1.81e+07	no	bt/maxit	0.000	0.07	3	1.81e+07	no	maxit	0.000	0.14
10 ⁵	rand5	1	1.82e+07	no	bt/maxit	0.000	0.07	3	1.82e+07	no	maxit	0.000	0.14

Table 28: Broyden31 – Tab k4_h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	6	$3.51e-09$	yes	–	1.979	0.01	7	$6.17e-11$	yes	–	1.992	0.02
2	rand1	7	$1.73e-09$	yes	–	1.982	0.00	8	$5.60e-10$	yes	–	1.992	0.00
2	rand2	6	$3.12e-12$	yes	–	2.014	0.00	7	$1.15e-08$	yes	–	1.991	0.00
2	rand3	7	$3.40e-13$	yes	–	1.990	0.00	8	$4.88e-10$	yes	–	1.992	0.00
2	rand4	6	$1.33e-07$	yes	–	2.011	0.00	7	$8.01e-08$	yes	–	1.987	0.00
2	rand5	7	$2.43e-10$	yes	–	2.005	0.00	6	$4.72e-08$	yes	–	1.981	0.00
10^3	xbar	13	$7.22e-07$	yes	–	0.815	0.07	13	$1.91e-07$	yes	–	1.185	0.01
10^3	rand1	14	$3.08e-07$	yes	–	1.050	0.07	15	$1.56e-07$	yes	–	0.879	0.00
10^3	rand2	14	$5.53e-07$	yes	–	1.015	0.06	15	$6.72e-07$	yes	–	0.820	0.00
10^3	rand3	14	$6.19e-07$	yes	–	0.860	0.06	15	$2.27e-07$	yes	–	1.016	0.00
10^3	rand4	14	$5.33e-07$	yes	–	0.877	0.07	15	$1.44e-07$	yes	–	1.154	0.00
10^3	rand5	15	$8.56e-08$	yes	–	0.887	0.07	15	$1.76e-07$	yes	–	0.952	0.00
10^4	xbar	13	$5.10e-07$	yes	–	1.036	1.47	13	$6.24e-07$	yes	–	0.793	0.03
10^4	rand1	16	$3.07e-07$	yes	–	0.942	1.89	16	$6.68e-07$	yes	–	0.936	0.03
10^4	rand2	16	$1.63e-07$	yes	–	0.893	1.77	16	$3.95e-07$	yes	–	0.824	0.05
10^4	rand3	15	$3.13e-07$	yes	–	0.907	1.74	16	$1.32e-07$	yes	–	1.028	0.04
10^4	rand4	16	$7.18e-07$	yes	–	1.012	1.85	17	$5.02e-07$	yes	–	0.891	0.04
10^4	rand5	16	$3.78e-07$	yes	–	0.951	1.83	16	$6.56e-07$	yes	–	0.906	0.04
10^5	xbar	4	$3.37e-09$	yes	–	1.782	1.06	5	$1.32e-07$	yes	–	1.990	0.08
10^5	rand1	6	$1.05e-09$	yes	–	1.982	1.46	6	$7.04e-09$	yes	–	1.987	0.00
10^5	rand2	6	$1.74e-12$	yes	–	1.933	1.54	6	$1.03e-12$	yes	–	1.993	0.00
10^5	rand3	6	$1.39e-10$	yes	–	1.967	1.45	6	$5.78e-08$	yes	–	1.991	0.00
10^5	rand4	6	$6.75e-13$	yes	–	1.925	1.62	7	$3.51e-07$	yes	–	1.981	0.00
10^5	rand5	6	$1.47e-10$	yes	–	1.985	1.59	7	$4.23e-13$	yes	–	1.997	0.00

Table 29: Broyden31 – Tab k4_hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	6	$3.51e-09$	yes	–	1.979	0.00	7	$6.17e-11$	yes	–	1.992	0.02
2	rand1	7	$1.73e-09$	yes	–	1.982	0.00	8	$5.60e-10$	yes	–	1.992	0.00
2	rand2	6	$3.12e-12$	yes	–	2.014	0.00	7	$1.15e-08$	yes	–	1.991	0.00
2	rand3	7	$3.39e-13$	yes	–	1.990	0.00	8	$4.88e-10$	yes	–	1.992	0.00
2	rand4	6	$1.33e-07$	yes	–	2.011	0.00	7	$8.01e-08$	yes	–	1.987	0.00
2	rand5	7	$2.43e-10$	yes	–	2.005	0.00	6	$4.72e-08$	yes	–	1.981	0.00
10^3	xbar	13	$7.22e-07$	yes	–	0.815	0.07	13	$1.91e-07$	yes	–	1.185	0.01
10^3	rand1	14	$3.08e-07$	yes	–	1.050	0.06	15	$1.56e-07$	yes	–	0.879	0.00
10^3	rand2	14	$5.53e-07$	yes	–	1.015	0.07	15	$6.72e-07$	yes	–	0.820	0.00
10^3	rand3	14	$6.19e-07$	yes	–	0.860	0.06	15	$2.27e-07$	yes	–	1.016	0.00
10^3	rand4	14	$5.33e-07$	yes	–	0.877	0.06	15	$1.44e-07$	yes	–	1.154	0.00
10^3	rand5	15	$8.56e-08$	yes	–	0.887	0.06	15	$1.76e-07$	yes	–	0.952	0.00
10^4	xbar	13	$5.10e-07$	yes	–	1.036	1.52	13	$6.24e-07$	yes	–	0.793	0.03
10^4	rand1	16	$3.07e-07$	yes	–	0.942	1.90	16	$6.68e-07$	yes	–	0.936	0.03
10^4	rand2	16	$1.63e-07$	yes	–	0.893	1.84	16	$3.95e-07$	yes	–	0.824	0.05
10^4	rand3	15	$3.13e-07$	yes	–	0.907	1.73	16	$1.32e-07$	yes	–	1.028	0.04
10^4	rand4	16	$7.18e-07$	yes	–	1.012	1.78	17	$5.02e-07$	yes	–	0.891	0.04
10^4	rand5	16	$3.78e-07$	yes	–	0.951	1.82	16	$6.56e-07$	yes	–	0.906	0.04
10^5	xbar	4	$3.37e-09$	yes	–	1.782	0.98	5	$1.32e-07$	yes	–	1.990	0.08
10^5	rand1	6	$1.05e-09$	yes	–	1.982	1.45	6	$7.04e-09$	yes	–	1.987	0.00
10^5	rand2	6	$1.74e-12$	yes	–	1.933	1.70	6	$1.03e-12$	yes	–	1.993	0.00
10^5	rand3	6	$1.39e-10$	yes	–	1.967	1.35	6	$5.78e-08$	yes	–	1.991	0.00
10^5	rand4	6	$6.71e-13$	yes	–	1.925	1.51	7	$3.51e-07$	yes	–	1.981	0.00
10^5	rand5	6	$1.47e-10$	yes	–	1.985	1.95	7	$4.23e-13$	yes	–	1.997	0.00

Table 30: Broyden31 – Tab k8_h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	6	$3.51e-09$	yes	–	1.979	0.00	7	$6.17e-11$	yes	–	1.992	0.02
2	rand1	7	$1.73e-09$	yes	–	1.982	0.00	8	$5.60e-10$	yes	–	1.992	0.00
2	rand2	6	$3.12e-12$	yes	–	2.015	0.00	7	$1.15e-08$	yes	–	1.991	0.00
2	rand3	7	$3.48e-13$	yes	–	1.987	0.00	8	$4.88e-10$	yes	–	1.992	0.00
2	rand4	6	$1.33e-07$	yes	–	2.011	0.00	7	$8.01e-08$	yes	–	1.987	0.00
2	rand5	7	$2.43e-10$	yes	–	2.005	0.00	6	$4.72e-08$	yes	–	1.981	0.00
10^3	xbar	13	$7.22e-07$	yes	–	0.815	0.05	13	$1.91e-07$	yes	–	1.185	0.01
10^3	rand1	14	$3.08e-07$	yes	–	1.050	0.06	15	$1.56e-07$	yes	–	0.879	0.00
10^3	rand2	14	$5.53e-07$	yes	–	1.015	0.07	15	$6.72e-07$	yes	–	0.820	0.00
10^3	rand3	14	$6.19e-07$	yes	–	0.860	0.07	15	$2.27e-07$	yes	–	1.016	0.00
10^3	rand4	14	$5.33e-07$	yes	–	0.877	0.07	15	$1.44e-07$	yes	–	1.154	0.00
10^3	rand5	15	$8.56e-08$	yes	–	0.887	0.07	15	$1.76e-07$	yes	–	0.952	0.00
10^4	xbar	13	$5.10e-07$	yes	–	1.036	1.55	13	$6.24e-07$	yes	–	0.793	0.03
10^4	rand1	16	$3.07e-07$	yes	–	0.942	1.89	16	$6.68e-07$	yes	–	0.936	0.03
10^4	rand2	16	$1.63e-07$	yes	–	0.893	1.87	16	$3.95e-07$	yes	–	0.824	0.05
10^4	rand3	15	$3.13e-07$	yes	–	0.907	1.70	16	$1.32e-07$	yes	–	1.028	0.04
10^4	rand4	16	$7.18e-07$	yes	–	1.012	1.76	17	$5.02e-07$	yes	–	0.891	0.04
10^4	rand5	16	$3.78e-07$	yes	–	0.951	1.77	16	$6.56e-07$	yes	–	0.906	0.04
10^5	xbar	4	$3.37e-09$	yes	–	1.782	1.00	5	$1.32e-07$	yes	–	1.990	0.08
10^5	rand1	6	$1.06e-09$	yes	–	1.982	1.52	6	$7.04e-09$	yes	–	1.987	0.00
10^5	rand2	6	$1.81e-12$	yes	–	1.928	1.46	6	$1.03e-12$	yes	–	1.993	0.00
10^5	rand3	6	$1.40e-10$	yes	–	1.967	1.47	6	$5.78e-08$	yes	–	1.991	0.00
10^5	rand4	6	$7.23e-13$	yes	–	1.916	1.47	7	$3.51e-07$	yes	–	1.981	0.00
10^5	rand5	6	$1.47e-10$	yes	–	1.984	1.53	7	$4.23e-13$	yes	–	1.997	0.00

Table 31: Broyden31 – Tab k8_hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	6	$3.51e-09$	yes	–	1.979	0.00	7	$6.17e-11$	yes	–	1.992	0.02
2	rand1	7	$1.73e-09$	yes	–	1.982	0.00	8	$5.60e-10$	yes	–	1.992	0.00
2	rand2	6	$3.11e-12$	yes	–	2.015	0.00	7	$1.15e-08$	yes	–	1.991	0.00
2	rand3	7	$3.42e-13$	yes	–	1.989	0.00	8	$4.88e-10$	yes	–	1.992	0.00
2	rand4	6	$1.33e-07$	yes	–	2.011	0.00	7	$8.01e-08$	yes	–	1.987	0.00
2	rand5	7	$2.43e-10$	yes	–	2.004	0.00	6	$4.72e-08$	yes	–	1.981	0.00
10^3	xbar	13	$7.22e-07$	yes	–	0.815	0.06	13	$1.91e-07$	yes	–	1.185	0.01
10^3	rand1	14	$3.08e-07$	yes	–	1.050	0.06	15	$1.56e-07$	yes	–	0.879	0.00
10^3	rand2	14	$5.53e-07$	yes	–	1.015	0.06	15	$6.72e-07$	yes	–	0.820	0.00
10^3	rand3	14	$6.19e-07$	yes	–	0.860	0.07	15	$2.27e-07$	yes	–	1.016	0.00
10^3	rand4	14	$5.33e-07$	yes	–	0.877	0.06	15	$1.44e-07$	yes	–	1.154	0.00
10^3	rand5	15	$8.56e-08$	yes	–	0.887	0.07	15	$1.76e-07$	yes	–	0.952	0.00
10^4	xbar	13	$5.10e-07$	yes	–	1.036	1.44	13	$6.24e-07$	yes	–	0.793	0.03
10^4	rand1	16	$3.07e-07$	yes	–	0.942	1.76	16	$6.68e-07$	yes	–	0.936	0.03
10^4	rand2	16	$1.63e-07$	yes	–	0.893	1.76	16	$3.95e-07$	yes	–	0.824	0.05
10^4	rand3	15	$3.13e-07$	yes	–	0.907	1.82	16	$1.32e-07$	yes	–	1.028	0.04
10^4	rand4	16	$7.18e-07$	yes	–	1.012	2.79	17	$5.02e-07$	yes	–	0.891	0.04
10^4	rand5	16	$3.78e-07$	yes	–	0.951	3.50	16	$6.56e-07$	yes	–	0.906	0.04
10^5	xbar	4	$3.37e-09$	yes	–	1.782	1.11	5	$1.32e-07$	yes	–	1.990	0.08
10^5	rand1	6	$1.05e-09$	yes	–	1.982	1.34	6	$7.04e-09$	yes	–	1.987	0.00
10^5	rand2	6	$1.77e-12$	yes	–	1.931	1.32	6	$1.03e-12$	yes	–	1.993	0.00
10^5	rand3	6	$1.40e-10$	yes	–	1.967	1.48	6	$5.78e-08$	yes	–	1.991	0.00
10^5	rand4	6	$6.98e-13$	yes	–	1.921	1.64	7	$3.51e-07$	yes	–	1.981	0.00
10^5	rand5	6	$1.47e-10$	yes	–	1.984	1.59	7	$4.23e-13$	yes	–	1.997	0.00

Table 32: Broyden31 – Tab k12_h: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	6	$1.37e-08$	yes	–	1.735	0.00	7	$6.17e-11$	yes	–	1.992	0.02
2	rand1	7	$1.66e-08$	yes	–	1.598	0.00	8	$5.60e-10$	yes	–	1.992	0.00
2	rand2	6	$5.36e-10$	yes	–	1.354	0.00	7	$1.15e-08$	yes	–	1.991	0.00
2	rand3	7	$1.42e-10$	yes	–	1.238	0.00	8	$4.88e-10$	yes	–	1.992	0.00
2	rand4	6	$7.03e-08$	yes	–	2.141	0.00	7	$8.01e-08$	yes	–	1.987	0.00
2	rand5	7	$5.03e-09$	yes	–	1.525	0.00	6	$4.72e-08$	yes	–	1.981	0.00
10^3	xbar	4	$3.32e-09$	yes	–	1.735	0.07	13	$1.91e-07$	yes	–	1.185	0.01
10^3	rand1	5	$2.23e-07$	yes	–	1.836	0.05	15	$1.56e-07$	yes	–	0.879	0.00
10^3	rand2	6	$8.23e-12$	yes	–	1.996	0.07	15	$6.72e-07$	yes	–	0.820	0.00
10^3	rand3	6	$1.79e-12$	yes	–	1.974	0.05	15	$2.27e-07$	yes	–	1.016	0.00
10^3	rand4	6	$3.32e-11$	yes	–	2.004	0.04	15	$1.44e-07$	yes	–	1.154	0.00
10^3	rand5	5	$1.18e-07$	yes	–	1.868	0.05	15	$1.76e-07$	yes	–	0.952	0.00
10^4	xbar	4	$3.37e-09$	yes	–	1.782	1.05	13	$6.24e-07$	yes	–	0.793	0.03
10^4	rand1	6	$1.05e-09$	yes	–	1.982	1.40	16	$6.68e-07$	yes	–	0.936	0.03
10^4	rand2	6	$1.74e-12$	yes	–	1.933	1.43	16	$3.95e-07$	yes	–	0.824	0.05
10^4	rand3	6	$1.39e-10$	yes	–	1.967	1.32	16	$1.32e-07$	yes	–	1.028	0.04
10^4	rand4	6	$6.75e-13$	yes	–	1.925	1.60	17	$5.02e-07$	yes	–	0.891	0.04
10^4	rand5	6	$1.47e-10$	yes	–	1.985	1.53	16	$6.56e-07$	yes	–	0.906	0.04

Table 33: Broyden31 – k12_hi: Modified Newton vs Truncated Newton with approximated gradient and hessian (CASE 2).

Dim	Start	Modified Newton						Truncated Newton					
		iter	$\ \nabla f\ $	success	flag	rate	time (s)	iter	$\ \nabla f\ $	success	flag	rate	time (s)
2	xbar	6	$1.37e-08$	yes	–	1.735	0.00	7	$6.17e-11$	yes	–	1.992	0.02
2	rand1	7	$1.07e-08$	yes	–	1.672	0.00	8	$5.60e-10$	yes	–	1.992	0.00
2	rand2	6	$2.60e-10$	yes	–	1.436	0.00	7	$1.15e-08$	yes	–	1.991	0.00
2	rand3	7	$3.12e-10$	yes	–	1.150	0.00	8	$4.88e-10$	yes	–	1.992	0.00
2	rand4	6	$7.02e-08$	yes	–	2.143	0.00	7	$8.01e-08$	yes	–	1.987	0.00
2	rand5	7	$3.95e-09$	yes	–	1.551	0.00	6	$4.72e-08$	yes	–	1.981	0.00
10^3	xbar	4	$4.05e-08$	yes	–	1.360	0.03	13	$1.91e-07$	yes	–	1.185	0.01
10^3	rand1	5	$3.05e-07$	yes	–	1.774	0.03	15	$1.56e-07$	yes	–	0.879	0.00
10^3	rand2	6	$2.36e-09$	yes	–	1.181	0.05	15	$6.72e-07$	yes	–	0.820	0.00
10^3	rand3	6	$1.13e-09$	yes	–	1.181	0.05	15	$2.27e-07$	yes	–	1.016	0.00
10^3	rand4	6	$2.04e-09$	yes	–	1.402	0.07	15	$1.44e-07$	yes	–	1.154	0.00
10^3	rand5	5	$3.17e-07$	yes	–	1.684	0.04	15	$1.76e-07$	yes	–	0.952	0.00
10^4	xbar	4	$8.01e-08$	yes	–	1.325	1.05	13	$6.24e-07$	yes	–	0.793	0.03
10^4	rand1	6	$1.73e-08$	yes	–	1.515	1.40	16	$6.68e-07$	yes	–	0.936	0.03
10^4	rand2	6	$1.80e-09$	yes	–	1.081	1.43	16	$3.95e-07$	yes	–	0.824	0.05
10^4	rand3	6	$6.81e-09$	yes	–	1.376	1.32	16	$1.32e-07$	yes	–	1.028	0.04
10^4	rand4	6	$7.43e-10$	yes	–	1.084	1.60	17	$5.02e-07$	yes	–	0.891	0.04
10^4	rand5	6	$1.12e-08$	yes	–	1.338	1.53	16	$6.56e-07$	yes	–	0.906	0.04

References

- [1] L. Luksan and J. Vlček, *Test Problems for Unconstrained Optimization*, Technical report, Nov. 2003. https://www.researchgate.net/publication/325314497_Test_Problems_for_Unconstrained_Optimization